

Tropical Monte Carlo v2

Comprehensive Reference Manual

*A numerical integration pipeline for convergent generalized Euler integrals
using tropical geometry and Monte Carlo sampling.*

Contents

1	Overview and Purpose	4
2	Mathematical Background	4
2.1	The Problem	4
2.2	Tropical Geometry Approach	5
2.3	Why This Helps	5
3	File Structure	5
4	Architecture: The 3-Stage Pipeline	6
4.1	Stage 1: Tropical Fan Decomposition (external code)	6
4.2	Stage 2: Symbolic Sector Processing	6
4.3	Stage 3: C++ Code Generation and Execution	7
4.4	Optional Stage: Auxiliary-Variable Lifting	7
5	Core File Reference	8
5.1	<code>tropical_fan.wl</code> — Tropical Fan Computation (external code, ≈ 470 lines)	8
5.2	<code>tropical_eval.wl</code> — Main Evaluation Package (≈ 2400 lines)	9
5.2.1	Module 1: <code>ProcessSector</code> (Symbolic Sector Processing)	9
5.2.2	Module 1b: Auxiliary-Variable Lifting	10
5.2.3	Module 2: Divergence Regulation [removed in v2]	11
5.2.4	Module 3: <code>MmaToC</code> and <code>GenerateCppMonteCarlo</code> (C++ Code Generation)	12
5.2.5	Module 4: <code>EvaluateTropicalMC</code> (Top-Level Driver)	13
5.2.6	Error Messages (<code>TropicalEval::*</code>)	14
6	Data Structures	15
6.1	<code>IntegrandSpec</code> (Mathematica Association)	15
6.2	<code>SectorData</code> (from <code>ProcessSector</code>)	15
6.3	<code>LiftData</code> (from <code>LiftCoefficients</code>)	16
6.4	Lifted <code>SectorData</code> extensions (from <code>ProcessSectorLifted</code>)	16
7	The Algorithm in Detail	16
7.1	Setup and Notation	16
7.2	Step 0: The Tropical Fan (computed by other code)	17
7.3	Step 1: Sector Decomposition by Monomial Maps	17
7.4	Step 2: Tropical Factoring (exact)	18
7.5	Step 3: Effective Exponents and the Convergence Gate	18
7.6	Step 4: Flattening	19
7.7	The Master Formula	19
7.8	Step 5: Uniform Monte Carlo and Error Estimation	19
7.9	Numerical Integrator: Monte Carlo vs. VEGAS	20
7.10	Validation Layer	22
7.11	The Lifted Algorithm: Delta Elimination in Detail	22
7.12	Worked Example: Why Lifting Works (Toy 0 in 1D)	23
7.13	Worked Example, step by step: a <i>small</i> coefficient becomes an $O(1)$ flattened integrand	25
7.14	Choosing the Lift Anchor z_0 (the integer k)	27
7.15	Log-Exp Polynomial Evaluation	29
7.16	Welford Online Variance Estimation	29
7.17	Treatment of Complex Exponents	30

7.18	Worked Example: Complex Flattening as a Contour Rotation	31
8	Dependencies	32
8.1	External Software	32
8.2	Installing Polymake	33
8.3	Internal Dependencies	34
9	Usage and Workflow	34
9.1	Minimal Convergent Integral	34
9.2	Kinematic Scan (Many Evaluation Points)	35
9.3	Extreme Coefficients via Lifting	35
9.4	Command-Line Execution	36
10	Examples Directory	37
11	Validation and Testing	39
11.1	6-Test Suite (<code>RunAllTests[]</code>)	39
11.2	Test 23: Lifted Pipeline (<code>test_lifted.wl</code>)	40
11.3	Test 24: Lifting with Complex, Small-Magnitude Coefficients (<code>test_complex_lifted.wl</code>)	40
11.4	Independent Cross-Checks: Exact Closed Form and CUBA (Examples 21–23) . .	41
12	Limitations and Caveats	41
13	Mathematical Properties and Why It Works	42

1 Overview and Purpose

The **Tropical Monte Carlo v2** project is a numerical integration pipeline for evaluating convergent generalized Euler integrals of the form:

$$I = \int_{[0,\infty)^n} dx_1 \cdots dx_n \prod_i x_i^{A_i} \prod_j P_j(\mathbf{x})^{B_j} \quad (1)$$

where:

- $P_j(\mathbf{x})$ are polynomials with real positive coefficients that may depend on kinematic parameters (e.g., masses, momenta in physics).
- A_i are the monomial exponents (may be complex).
- B_j are the polynomial exponents (typically negative, may be complex).

These integrals arise in quantum field theory (Feynman integrals), cosmological correlator calculations, and precision physics. They are often high-dimensional, and standard numerical quadrature fails for large n .

Scope of v2.

This version supports **convergent integrals only**: the `RegulatorSymbol` field of the integrand specification must be `None` (or absent), and every sector of the tropical decomposition must have effective exponents with positive real part. Specifications that violate either condition are rejected with the error messages `TropicalEval::noregulator` and `TropicalEval::divergentinput`. For ε -regulated or divergent integrals (pole extraction by tropical subtraction or integration by parts), use the original `TROPICAL_MONTE_CARLO` package, which retains that machinery.

Key Innovation.

The code uses **tropical geometry**—specifically the Newton polytope fan decomposition—to partition the integration domain into simplicial sectors. Within each sector, symbolic coordinate transformations and a “flattening” procedure render the integrand $\mathcal{O}(1)$ on the unit hypercube $[0, 1]^n$, making uniform Monte Carlo sampling highly efficient.

A single C++ compilation handles thousands of kinematic points in parallel via OpenMP, since the fan decomposition depends only on the exponent structure, not on the polynomial coefficients.

Variance control for extreme coefficients.

When a polynomial coefficient is many orders of magnitude larger or smaller than the others, the tropical decomposition remains *exact*, but the Monte Carlo variance in some sectors can grow substantially. v2 adds an **auxiliary-variable lifting** module (Section 5.2.2) that replaces an extreme coefficient C by a power z^k of a new variable anchored at $z_0 = |C|^{1/k}$, resolves the resulting delta constraint analytically, and evaluates the lifted integrand on the $(n+1)$ -dimensional tropical fan. In benchmarked cases this reduces the per-sample standard deviation by an order of magnitude or more.

2 Mathematical Background

2.1 The Problem

We want to compute:

$$I(\lambda) = \int_{[0,\infty)^n} \prod_i x_i^{A_i} \prod_j P_j(\mathbf{x}; \lambda)^{B_j} d\mathbf{x} \quad (2)$$

for many values of kinematic parameters $\lambda = (\lambda_1, \lambda_2, \dots)$.

Direct Monte Carlo on $[0, \infty)^n$ is impractical because:

- The integrand has singular behavior near coordinate hyperplanes ($x_i \rightarrow 0$).
- Different monomials dominate in different regions.
- Variance is enormous without importance sampling.

2.2 Tropical Geometry Approach

The tropical version of a polynomial $P(\mathbf{x}) = \sum_m c_m \mathbf{x}^m$ replaces multiplication with addition and addition with min:

$$\text{trop}(P)(\mathbf{w}) = \min_m \{m \cdot \mathbf{w}\} \quad (\mathbf{w} \in \mathbb{R}^n) \quad (3)$$

The set of $\mathbf{w}$ where the minimum is achieved by multiple monomials defines the *tropical variety*. The **Newton polytope** of P is the convex hull of the exponent vectors $\{m\}$. Its **normal fan** partitions $\mathbb{R}^n$ into cones, each corresponding to a region where one monomial dominates.

2.3 Why This Helps

In each cone (sector), the dominant monomial can be factored out symbolically. After a monomial change of variables aligned with the cone’s rays, the remaining polynomial has no singular behavior—it equals $1 + (\text{bounded terms})$. A further “flattening” coordinate change absorbs the monomial singularity x^{A-1} into the measure, producing an $\mathcal{O}(1)$ integrand on $[0, 1]^n$.

3 File Structure

```
TROPICAL_MONTE_CARLOv2/
|
|-- SUMMARY.txt           Project documentation
|-- tropical_fan.wl       Tropical fan computation (~470 lines;
|                           separate code, black box in this manual)
|-- tropical_eval.wl      Main evaluation package (~2400 lines)
|
|-- EXAMPLES/
|   |-- tropical_eval_examples.wl Examples 1-14 (+ defines RunAllTests suite)
|   |-- run_validation_suite.wl Examples 1-14 then invokes RunAllTests[]
|   |-- tropical_eval_examples2.wl Examples 15-20: complex exponents,
|   |                               small/extreme coefficients, lifting
|   |-- tropical_eval_examples3.wl Examples 21-23: independent cross-checks
|   |                               (exact Gamma closed form + CUBA library)
|   |-- cuba_common.wl       Shared CUBA cross-check generator
|   |-- tropical_fan_examples.wl Fan computation examples
|   |-- test_quick.wl        Minimal smoke test (Example 1)
|   |-- test_cpp.wl          C++ codegen & execution test
|   |-- test_seedbase.wl     SeedBase runtime-argv override test
|   |-- test_lifted.wl       Test 23: lifted-integral pipeline
|   |-- test_cuba.wl         CUBA cross-check of all test integrals
|   |-- test_vegas.wl        MC vs VEGAS parity (requires CUBA)
|   |-- prompt_tropical_mc.txt Original development specification
|
|-- SANDBOX/               Lifting development experiments
```

```

| |-- bench_lift_variance.wl Variance-reduction benchmark
| |-- benchmark_results.md  Benchmark tables (Cases A/B/C)
| |-- sandbox_toy*.wl       Toy-model lifting studies
|
|-- CC/                      Historical cross-checks vs FIESTA
|                             (from the original package; scripts that
|                             call removed divergent/IBP functions are
|                             NOT runnable against v2)
|
|-- INTERFILES/              Generated intermediate files
| |-- .gitignore             (all contents gitignored)
| |-- tropical_mc_generated.cpp Generated C++ source
| |-- tropical_mc            Compiled binary
| |-- kinematic_data.txt     Input kinematic file
| |-- mc_results.txt         MC output results
|
|-- MANUAL/                  This manual

```

4 Architecture: The 3-Stage Pipeline

The code operates as a 3-stage pipeline (plus an optional lifting stage for integrands with extreme coefficients):

4.1 Stage 1: Tropical Fan Decomposition (external code)

Source:

`tropical_fan.wl` — separate code, treated as a black box in this manual

Input:

Newton polytope of $P_1 \cdot P_2 \cdots P_m$ (its lattice points)

Output:

`{dualVertices, simplexList}`: integer ray generators plus simplicial cones (list of ray-index n -tuples)

The fan construction itself (convex hull $\rightarrow$ normal fan $\rightarrow$ simplicial triangulation, performed via the external tool Polymake) is *not* part of the evaluation algorithm described in this manual; Section 7.2 states the properties the evaluation pipeline relies on. Any fan with those properties can be supplied by hand in place of the computed one.

4.2 Stage 2: Symbolic Sector Processing

Source:

`tropical_eval.wl`, Module 1

Input:

Fan decomposition + `IntegrandSpec`

Process:

For each simplicial cone:

- (a) Build ray matrix M , check $\det(M) \neq 0$
- (b) Monomial change of variables $\mathbf{x} \rightarrow \mathbf{y}$ via M
- (c) Tropical factoring (extract dominant monomials)

- (d) Compute effective exponents a_{eff}
- (e) Flattening substitution $\mathbf{y} \rightarrow \mathbf{y}'$ (absorb singularity)
- (f) Divide exponents, compute prefactor

Output:

`SectorData` association per cone (13+ fields)

Convergence guard.

A sector with $\text{Re}(a_{\text{eff},k}) \leq 0$ for some k is flagged `IsDivergent`. In v2 such sectors are *not* processed further: the driver fires `TropicalEval::divergentinput` and returns `$Failed`. (The original package's divergence-regulation machinery—tropical subtraction and IBP pole extraction, formerly Modules 2 and 2b—was removed in v2.)

4.3 Stage 3: C++ Code Generation and Execution**Source:**

`tropical_eval.wl`, Modules 3–4

Input:

Convergent sector data + kinematic points

Process: (a) Translate Mathematica expressions to C++ strings

- (b) Generate one integrand function per sector (`integrand_conv_N`)
- (c) Generate `main()` with file I/O, OpenMP, Welford statistics
- (d) Compile with `g++ -std=c++17` (auto-detect OpenMP)
- (e) Execute binary, read results from output file

Output:

Results (mean $\pm$ error per kinematic point)

4.4 Optional Stage: Auxiliary-Variable Lifting**Source:**

`tropical_eval.wl`, Modules 1b and 4b

Input:

`IntegrandSpec` with one or more extreme coefficients

Process: (a) Detect extreme coefficients (`DetectExtremeCoefficients`)

- (b) Replace coefficient $C \mathbf{x}^\alpha \rightarrow z^k \mathbf{x}^\alpha$ with anchor $z_0 = |C|^{1/k}$ (`LiftCoefficients`)
- (c) Build the $(n+1)$ -dimensional fan of the lifted polynomial
- (d) Per sector: resolve the $\delta(z - z_0)$ constraint analytically via a pivot substitution (`ProcessSectorLifted`), reducing back to dimension n
- (e) Continue with Stage 3 on the lifted sectors (domain-constrained integrands)

Output:

Same result format, often with substantially reduced MC variance

5 Core File Reference

5.1 `tropical_fan.wl` — Tropical Fan Computation (external code, ≈ 470 lines)

Role.

This file is *other code* whose only job is to actually compute the tropical fan — the simplicial normal fan of the Newton polytope of the product polynomial. **This manual treats it as a black box:** the geometric construction (convex hulls, normal fans, triangulation, all performed by calling the external tool Polymake) is not described here. Everything the evaluation package needs from it is the pair

```
fanData = {dualVertices, simplexList}
```

where `dualVertices` is a list of integer ray vectors and `simplexList` a list of n -tuples of ray indices, one tuple per simplicial cone. The properties this output must satisfy are spelled out in Section 7.2; any fan with those properties (including a hand-written one, as in the lifted tests 23A/23D) is a valid drop-in replacement.

Public interface used by the pipeline:

`PolytopeVertices[integrand, vars]`

Extracts the exponent vectors (Newton polytope lattice points) from the integrand expression.

`ComputeDecomposition[vertices, "ShowProgress" -> True]`

Computes the fan; returns `{dualVertices, simplexList}`.

`ComputeDecompositiony[vertices]`

Variant that intersects the fan with the positive orthant (special-purpose).

`$TropicalFanPolymakePath`

Set before loading to override the Polymake auto-detection (`/opt/homebrew`, `/usr/local`, `/usr`, or `PATH`).

`TropicalFan::polymake`

Message fired when the underlying Polymake call fails.

Skipping the dependency.

Setting the flag `$SkipPolymakeLoad` to `True` *before* loading `tropical_eval.wl` skips loading `tropical_fan.wl` entirely (useful on machines without Polymake, e.g. cluster nodes). The evaluation pipeline then runs against explicitly supplied `fanData`; only `PolytopeVertices` / `ComputeDecomposition` become unavailable.

Typical Output.

For a 2D polynomial with 4 terms:

```
dualVertices = {{1,0}, {0,1}, {-1,0}, {0,-1}, {1,1}, ...}
simplexList = {{1,2}, {2,3}, {3,4}, {4,5}, ...}
```


5.2 tropical_eval.wl — Main Evaluation Package (≈ 2400 lines)

5.2.1 Module 1: ProcessSector (Symbolic Sector Processing)

Purpose.

Perform the symbolic coordinate transformation for one simplicial cone, converting the integrand into a form suitable for Monte Carlo sampling on $[0, 1]^n$.

Function Signature:

```
ProcessSector[integrandSpec, dualVertices, simplex, coneIndex]
```

Algorithm:

1. **Extract rays and build matrix.** Take the n rays for this cone and build $M = -\text{Transpose}[\{\rho_1, \dots, \rho_n\}]$. The change of variables is $x_i = \prod_j y_j^{M_{ij}}$.
2. **Check non-degeneracy.** Verify $\det(M) \neq 0$. Degenerate cones are skipped.
3. **Transform polynomials.** Substitute $\mathbf{x} \rightarrow \mathbf{y}^M$ in all polynomials: each monomial $c \cdot \mathbf{x}^\alpha$ becomes $c \cdot \mathbf{y}^{\alpha \cdot M}$.
4. **Tropical factoring.** For each polynomial $P_k(\mathbf{y})$, find

$$d_{k,j} = \min_m (\text{exponent of } y_j \text{ in monomial } m \text{ of } P_k) \quad (4)$$

Factor out $\mathbf{y}^{d_k}$ to get $Q_k(\mathbf{y})$ with all non-negative exponents. Q_k always contains a constant term (the dominant monomial).

5. **Compute effective exponents:**

$$a_{\text{eff},i} = \text{raw}A_i + \sum_k B_k \cdot d_{k,i} \quad (5)$$

6. **Check convergence.** If $\text{Re}(a_{\text{eff},i}) > 0$ for all i , the sector is convergent. Otherwise, flag as divergent (in v2 a divergent sector causes the driver to abort with `TropicalEval::divergentinput`).
7. **Flatten.** Substitute $y_i \rightarrow (y'_i)^{1/a_{\text{eff},i}}$. New exponents become $e_{m,i}/a_{\text{eff},i}$.
8. **Compute prefactor:**

$$\text{prefactor} = \frac{|\det(M)|}{\prod_i a_{\text{eff},i}} \quad (6)$$

Output.

A `SectorData` association with 13+ fields (see Section 6.2).

Related diagnostics.

```
CheckFlatteningMagnitude[sectorData, nSamples:20, testKinematics:{}]
```

spot-checks the flattened integrand magnitude at random points, warning when $\max > 10^3$ or $\min < 10^{-6}$ (a heavy-tail indicator); for lifted sectors carrying a `DomainConstraint` it draws feasible points by rejection sampling and additionally returns a `"FeasibleFraction"` key. `ValidateDecomposition` (Section 7.10) checks the whole decomposition against direct `NIntegrate`.

5.2.2 Module 1b: Auxiliary-Variable Lifting

Purpose.

Reduce Monte Carlo variance for integrands whose polynomial coefficients span many orders of magnitude (very large *or* very small coefficients). The tropical factoring is exact regardless of coefficient values, but when the tropically dominant monomial is not the numerically dominant one, the cleared polynomial Q can be far from $\mathcal{O}(1)$ on parts of $[0, 1]^n$, inflating the per-sample variance. Lifting moves the extreme coefficient into the geometry, where the fan can see it. Section 7.12 works this out, with numbers, on a one-dimensional example.

Idea.

Replace the extreme coefficient C in monomial $C \mathbf{x}^\alpha$ by z^k for a new auxiliary variable z , with anchor $z_0 = |C|^{1/k}$ so that $z = z_0$ recovers the original polynomial exactly:

$$I = \int_0^\infty dz \delta(z - z_0) \int_{[0, \infty)^n} d\mathbf{x} \text{ [lifted integrand]} \quad (7)$$

The $(n+1)$ -dimensional lifted integrand is decomposed on the tropical fan of the lifted polynomial; in each sector the delta constraint is resolved analytically by solving for a pivot variable (“delta elimination”), reducing the dimension back to n . Residual coefficients $c_i = C_i/z_0^{k_i}$ keep the construction exact when several monomials are lifted at once.

Complex coefficients.

The coefficient C being lifted may itself be *complex*: the anchor $z_0 = |C|^{1/k}$ uses the modulus and is therefore real and positive, while the entire phase is carried, exactly, by the residual $c = C/z_0^k = e^{i \arg C}$ (unit modulus for the primary rule, $\mathcal{O}(1)$ for the secondaries). The lifting thus splits a small *complex* coefficient cleanly into a real magnitude scale, which moves into the deterministic geometry/prefactor, and an $\mathcal{O}(1)$ complex phase, which rides along on the lifted monomial. Every quantity that must stay real to define the sector — the effective exponents a_{eff} , the convergence gate, and the delta-root domain indicator — depends only on the integer Newton-polytope data and the *real* exponents A_i, B_k , so it is unaffected; only the polynomial *values* become complex, and the C++ pipeline already carries those as `std::complex<double>`. The round-trip identity check in `LiftCoefficients` therefore holds exactly for complex C , and the lifted decomposition reproduces the (complex-valued) integral. Worked demonstrations with coefficients of very small magnitude are Examples 24–27 (Section 10); the boundary case — complex *exponents* B_k , which are *not* liftable — is contrasted there and below.

Key Functions:

DetectExtremeCoefficients[spec, threshold:1000, opts] Scans all polynomial monomials (via `ParsePolynomial`) and returns a list of associations `<|"PolyIndex", "ExponentVector", "Coefficient", "Magnitude", "SuggestedK"|>` for every monomial whose numeric coefficient satisfies $|C| < 1/\text{threshold}$ or $|C| > \text{threshold}$. `SuggestedK` automates the anchor $z_0 = |C|^{1/k}$: the `"AnchorRule"` option (default `"kStar"`) returns $k^* = \max(1, \lceil |\log |C|| / \log \text{threshold} \rceil)$ of Section 7.14; `"Unit"` restores the legacy `SuggestedK` $\rightarrow 1$, and a function `f[mag, threshold]` supplies a custom rule. The optional `"BandEdgeGuard"` (default `False`) adds +1 for coefficients within one decade of the band edge. Symbolic (kinematic) coefficients are skipped, since their magnitude is unknown at lift time.

LiftCoefficients[spec, liftRules] Applies the lifting. Each rule is `<|"PolyIndex"->j, "ExponentVector"->α, "k"->k|>`. Returns `<|"LiftedSpec"->..., "LiftData"->...|>` where `LiftData` holds the anchor `z0` (exact), the auxiliary variable `x[n+1]`, the residual coefficients, and a copy of the original spec. A round-trip identity check (lifted polynomial at $z \rightarrow z_0$ equals the original) fires `TropicalEval::liftidentity` on failure.

ProcessSectorLifted[*liftedSpec*, *dualVertices*, *simplex*, *coneIndex*, *liftData*]

Runs `ProcessSector` on the $(n+1)$ -dimensional lifted problem, then resolves the delta constraint: it selects a pivot variable p with z -row exponent $m_p \neq 0$, substitutes

$$y_p = z_0^{1/m_p} \prod_{j \neq p} y_j^{-m_j/m_p}, \quad (8)$$

re-clears the resulting polynomial, and returns an n -dimensional `SectorData` augmented with `DomainConstraint` (the region of $[0,1]^n$ where the pivot value lies in $(0,1]$), `PivotIndex`, `ZRow`, `AugmentedA`, and `HasConstantTerm`. Sectors whose delta root lies entirely outside the unit cube return `<|"EmptyDomain" -> True|>` and contribute zero. Pivot ranking: (1) prefer pivots that preserve a constant term after re-clearing, (2) prefer $|m_p| = 1$, (3) maximize the worst integrability margin $\min_i \operatorname{Re} \tilde{a}_i$. Error paths: `liftnopivot`, `liftcomplex`, `liftdivergent`.

ValidateLiftedDecomposition[*origSpec*, *liftedSpec*, *liftedFan*, *liftData*, *kin*, *pg*]

Cross-checks the lifted sector sum (per-sector `NIntegrate` over the constrained unit cube) against direct `NIntegrate` of the *original* integrand. Returns `DirectResult`, `SectorSum`, `RelativeError`, `SectorResults`, and `DroppedSectors`.

EvaluateTropicalMCLifted[*spec*, *kinPoints*, *opts*] Convenience driver: detect (or accept) lift rules, lift, build the lifted fan, and call `EvaluateTropicalMC` with `"LiftData"` set. Options: `"LiftRules"` (`Automatic` or explicit list), `"Threshold"` (default 1000), `"AnchorRule"` / `"BandEdgeGuard"` (passed to `DetectExtremeCoefficients` when `"LiftRules"` $\rightarrow$ `Automatic`, so the automatic path lifts at k^* rather than $k = 1$; Section 7.14), `"FanData"` (`Automatic` or an explicit lifted fan; the automatic path is now scale-robust — it retries the normal-fan computation on a scaled copy of the polytope, which is required for ambient dimension ≥ 4 where thin lattice simplices otherwise make the raw call fail — so an explicit fan is needed only when the lifted polytope is genuinely lower-dimensional, cf. `TropicalEval::liftdegenerate`), plus all `EvaluateTropicalMC` options. If automatic detection finds no extreme coefficients it falls back to the plain pipeline.

Caveats.

Lifting requires *real* polynomial exponents B_j — a complex $\tilde{a}$ admits no real-valued domain indicator, and `liftcomplex` fires otherwise. (This is a restriction on the *exponents*, not the coefficients: complex *coefficients* C are fully supported, as just described.) Complex coefficients are integrated on the principal branch of $\log Q$, which is the correct, `NIntegrate`-consistent value as long as no cleared polynomial Q_k winds around the origin on $[0,1]^n$; this is automatic when the dominant constant term keeps Q_k near the positive real axis (e.g. a small complex perturbation of a positive polynomial), and is exactly the regime where lifting is wanted. When the lifted Newton polytope is lower-dimensional (few distinct monomials), the automatic fan fails and an explicit `"FanData"` must be supplied. Sectors with `HasConstantTerm` $\rightarrow$ `False` may have infinite true variance: the sample-based error bar can then underestimate the true error. See `SUMMARY.txt` §8 and `SANDBOX/benchmark_results.md` for benchmark tables.

5.2.3 Module 2: Divergence Regulation [removed in v2]

Modules 2 and 2b were removed in `TROPICAL_MONTE_CARLOv2`. Module 2 (tropical subtraction) provided `IdentifyDivergences`, `ProcessDivergentSector`, and `ValidateSubtraction`; Module 2b (IBP pole extraction) provided `IBPReduceSector`, `IBPProcessSector`, `IBPCheckBoundary`, `ValidateIBP`, `GenerateCppMonteCarloIBP`, and `EvaluateTropicalMCIBP`. The v2 package evaluates convergent integrals only; divergent or ε -regulated inputs are rejected with

`TropicalEval::noregulator` or `TropicalEval::divergentinput`. For pole extraction, use the original `TROPICAL_MONTE_CARLO` package.

5.2.4 Module 3: `MmaToC` and `GenerateCppMonteCarlo` (C++ Code Generation)

`MmaToC[expr, kinematicSymbols]`.

Recursive pattern-matching converter from Mathematica to C++:

Mathematica	C++
Integer n	<code>n.0</code>
Rational p/q	<code>(p.0/q.0)</code>
Real r	CForm literal
<code>Complex[a,b]</code>	<code>cx(a, b)</code>
<code>Power[x, -1]</code>	<code>(1.0/x)</code>
<code>Power[x, 1/2] / Power[x, -1/2]</code>	<code>std::sqrt(x) / (1.0/std::sqrt(x))</code>
<code>Power[E, z], Exp[z]</code>	<code>std::exp(z)</code>
<code>Power[x, e]</code> (general)	<code>std::pow(x, e)</code>
<code>Log[x]</code>	<code>std::log(x)</code>
<code>Abs[x]</code>	<code>std::abs(x)</code>
<code>Re[z] / Im[z]</code>	<code>(z).real() / (z).imag()</code>
<code>Plus[a, b, ...]</code>	<code>(a + b + ...)</code>
<code>Times[a, b, ...]</code>	<code>(a * b * ...)</code>
Kinematic symbol	<code>params[i]</code> (via the parameter map)

Anything else falls through to a CForm-based fallback (with `Power/Sqrt` rewritten); after generation the full C++ source is scanned for unconverted Mathematica heads (`Sin`, `Power`, ...) and a warning is printed if any survive. Integration variables never pass through `MmaToC`: the polynomial evaluation is emitted directly from the parsed `{coefficient, exponent-vector}` data as `log_y[]` sums.

Polynomial evaluation uses the **log-exp trick**:

$$c \cdot y_1^{\alpha_1} \cdot y_2^{\alpha_2} = c \cdot \exp(\alpha_1 \log(y_1) + \alpha_2 \log(y_2)) \quad (9)$$

`GenerateCppMonteCarlo[convergentSectors, {}, spec, outputFile, opts]`.

Generates a complete, self-contained C++ source file with:

- **One integrand function per sector** (v2, convergent only):

integrand_conv_N: Returns $\text{prefactor} \times \prod_j Q_j(\mathbf{y})^{B_j}$, evaluated via the log-exp pattern with a boundary guard (`log_y[i] = (y[i] > 1e-300) ? std::log(y[i]) : -700.0`). For lifted sectors the function first evaluates the domain-constraint indicator and returns 0 outside the feasible region.

- **Function table**: arrays `integrand_table[]` (function pointers) and `integrand_dim[]` (per-sector dimensions) for dispatch.
- **main() function**: file I/O for kinematic data, OpenMP parallelization over kinematic points (`schedule(dynamic)`), per-point `std::mt19937_64` seeded with `SeedBase + kp` (deterministic, thread-count independent), Welford online statistics, optional debug mode with NaN detection and per-sector diagnostics (`-DTROPICAL_MC_DEBUG`).
- **Binary interface**: `./tropical_mc <input_file> <output_file> [n_samples] [n_threads]` (defaults: the baked-in "NSamples"; with OpenMP, `n_threads = 1` means all available threads).

- **Output format:** one line per kinematic point, 17 significant digits, columns `re`, `im`, `err_re`, `err_im`.

Options:

"`NSamples`" (default 10^6 ; the binary's default sample count), "`MaxDim`" (default 20; size of the static sample buffer — the per-sector dimension must not exceed it), "`SeedBase`" (default 42; RNG seed offset).

Compilation (`CompileCpp[src, bin, debug]`):

```
g++ -std=c++17 -O3 -fopenmp -Wall -Wextra -o bin src.cpp -lm # release
g++ -std=c++17 -O2 -fopenmp -DTROPICAL_MC_DEBUG ... # debug
```

Auto-retries without `-fopenmp` if compilation fails because of it (e.g., on macOS with clang).

5.2.5 Module 4: `EvaluateTropicalMC` (Top-Level Driver)

Purpose.

Orchestrate the entire pipeline from `IntegrandSpec` to final numerical results.

Function Signature:

```
EvaluateTropicalMC[integrandSpec, fanData, kinematicPoints, opts...]
```

Algorithm:

1. Validate inputs: reject any `RegulatorSymbol` other than `None` (`noregulator`); in lifted mode check the fan is $(n+1)$ -dimensional (`liftfandim`); check kinematic points are well-formed (correct length, real, finite).
2. `ProcessSector` for all simplicial cones in the fan (or `ProcessSectorLifted` when the "`LiftData`" option is set; `EmptyDomain` sectors are dropped and counted, and any `$Failed` aborts the call).
3. (If `RunChecks` and kinematic symbols are present) `ValidateDecomposition` / `ValidateLiftedDecomposition` at up to 3 kinematic points. For parameter-free integrands this step is skipped — call the validators directly instead.
4. Abort with `TropicalEval::divergentinput` if any sector is divergent.
5. `GenerateCppMonteCarlo` (write C++ source).
6. (If `RunChecks`) debug build (`-O2 -DTROPICAL_MC_DEBUG`) and a short test run (≤ 5 points, 10^5 samples, 2 threads) with NaN detection.
7. Write the kinematic data file; release compile (`-O3`); run the binary with `NSamples` and `NThreads` (`Automatic` $\rightarrow$ `$ProcessorCount`).
8. Parse the result file, assemble one association per kinematic point (rows failing to parse yield zero-filled entries with a warning).
9. Print an error summary (mean/median/max error bars, flagging points whose `ReErr` exceeds 10% of the magnitude); (if `RunChecks` and kinematic symbols are present) `NIntegrate` cross-check at up to 3 points.

Options:

Option	Default	Description
"NSamples"	1 000 000	MC samples per sector
"NThreads"	Automatic	OpenMP threads (\$ProcessorCount)
"RunChecks"	True	Debug build + NIntegrate cross-checks
"PrecisionGoal"	3	Precision goal for validation integrals
"WorkingDirectory"	Automatic	Where generated files go (INTERFILES/)
"Verbose"	True	Progress output
"LiftData"	None	Lift-data association; routes processing through ProcessSectorLifted (first argument is then the <i>lifted</i> spec)

Return value.

An association with keys "Results" (a list of <|"KinematicPoint", "Re", "Im", "ReErr", "ImErr"|> associations, one per kinematic point), "ConvergentSectors", "CppFile", and "ResultFile".

5.2.6 Error Messages (TropicalEval::*)

Message	Fired by	Meaning
degenerate	ProcessSector	Cone with $\det(M) = 0$; sector returns \$Failed.
notsimplicial	ProcessSector	Cone has $\neq n$ rays.
noregulator	ProcessSector, driver	RegulatorSymbol is not None; v2 is convergent-only.
divergentinput	driver, validators	Some sector fails the convergence gate $\text{Re } a_{\text{eff}} > 0$.
nodivergent	GenerateCppMonteCarlo	A non-empty divergent-sector list was passed (unsupported in v2).
validate	validators	Sector sum deviates from direct NIntegrate beyond 10^{-pg+1} .
liftidentity	LiftCoefficients	Round-trip check failed: lifted polynomial at $z \rightarrow z_0 \neq$ original.
liftnopivot	ProcessSectorLifted	No admissible pivot (message lists per pivot $\tilde{a}$).
liftcomplex	ProcessSectorLifted	Candidate pivots give complex $\tilde{a}$; lifting requires real B_k .
liftdivergent	ProcessSectorLifted	Selected sector divergent after delta resolution (safety guard).
liftfandim	EvaluateTropicalMC	Fan supplied in lifted mode is not $(n+1)$ -dimensional.
liftdegenerate	EvaluateTropicalMCLifted	Lifted Newton polytope is lower-dimensional; supply explicit "FanData".

(divergent, nested, and badck are also defined for compatibility with the original package but are never fired by the v2 pipeline.)

6 Data Structures

6.1 IntegrandSpec (Mathematica Association)

Key	Description
"Polynomials"	$\{P_1, P_2, \dots\}$ — symbolic polynomial expressions. Coefficients may depend on kinematic symbols.
"MonomialExponents"	$\{A_1, \dots, A_n\}$ — one per integration variable. May be numeric, symbolic in ε , or complex.
"PolynomialExponents"	$\{B_1, \dots, B_m\}$ — one per polynomial. Typically negative. May depend on ε .
"Variables"	$\{x_1, \dots, x_n\}$ — integration variable symbols.
"KinematicSymbols"	$\{\lambda, \mu, \dots\}$ or $\{\}$ — parameters varying across evaluation points.
"RegulatorSymbol"	Must be <code>None</code> (or absent) in v2. Any other value is rejected with <code>TropicalEval::noregulator</code> .

6.2 SectorData (from ProcessSector)

Key	Description
"ConeIndex"	Integer label for this simplicial cone.
"RayMatrix"	M ($n \times n$). Defines the change of variables $\mathbf{x} = \mathbf{y}^M$.
"DetM"	$\det(M)$. $ \det(M) $ appears in the Jacobian.
"SelectedRays"	$\{\rho_1, \dots, \rho_n\}$ — ray vectors defining this cone.
"RawExponents"	Exponents after the M -transformation, before tropical factoring.
"NewExponents"	Effective exponents: $a_{\text{eff},i} = \text{raw}A_i + \sum_k B_k d_{k,i}$.
"MinExponents"	$\{d_{k,j}\}$ per polynomial. Minimum exponent of y_j across monomials of P_k .
"TransformedPolys"	Monomials in $\mathbf{y}$ (exponents may be negative).
"ClearedPolys"	After tropical factoring. All exponents ≥ 0 .
"FlattenedPolys"	After flattening. Exponents are $e_{m,i}/a_{\text{eff},i}$.
"Prefactor"	$ \det(M) / \prod_i a_{\text{eff},i}$. May depend on ε .
"IsDivergent"	True or False.
"DivergentVariable"	Index k (1-based) or 0 if convergent.
"Dimension"	n , number of integration variables.
"PolynomialExponents"	$\{B_1, \dots\}$, copied from <code>IntegrandSpec</code> .
"MonomialExponents"	$\{A_1, \dots\}$, copied from <code>IntegrandSpec</code> .

For a *divergent* sector (`IsDivergent -> True`) the association contains the pre-flattening data only: there is no `FlattenedPolys` key and `Prefactor` holds just $|\det(M)|$. (The lifted path consumes this pre-flattened form before its own delta resolution.)

6.3 LiftData (from LiftCoefficients)

Key	Description
"z0"	The anchor value $z_0 = C_{\text{primary}} ^{1/k}$, stored exactly.
"AuxVariable"	The auxiliary variable symbol, $\mathbf{x}[\mathbf{n}+1]$.
"AuxIndex"	Index of the auxiliary variable, $n + 1$.
"Rules"	The list of lift rules applied.
"Residuals"	Residual coefficients $c_i = C_i/z_0^{k_i}$ (exactness for multi-rule lifts).
"OriginalSpec"	Copy of the unlifted <code>IntegrandSpec</code> .

6.4 Lifted SectorData extensions (from ProcessSectorLifted)

`ProcessSectorLifted` returns a standard n -dimensional convergent `SectorData` (after delta resolution) extended with:

Key	Description
"DomainConstraint"	Description of the feasible region of $[0, 1]^n$ (where the delta root for the pivot variable lies in $(0, 1]$); compiled into an indicator in the C++ integrand.
"PivotIndex"	The pivot variable p eliminated against the delta constraint.
"ZRow"	The z -row m of the ray matrix M (exponents of the auxiliary variable).
"AugmentedA"	Effective exponents of the $(n+1)$ -dimensional problem before delta resolution.
"HasConstantTerm"	Whether the re-cleared polynomial retains a constant term. <code>False</code> signals possible heavy tails (sample variance may be underestimated).
"EmptyDomain"	(Alternative return) <code>< "EmptyDomain"->True, "ConeIndex"->... ></code> when the delta root lies outside the unit cube; the sector contributes zero and is dropped.

7 The Algorithm in Detail

This section is a complete, self-contained description of the algorithm, from the input integral to the final Monte Carlo estimate with its error bar. The *only* step treated as a black box is the construction of the tropical fan itself: that computation is performed by other code (`tropical_fan.wl`, which calls Polymake) and is outside the scope of this manual. Section 7.2 states exactly what the evaluation algorithm assumes about the fan it is handed; everything downstream of that point is described in full.

7.1 Setup and Notation

The input is a convergent generalized Euler integral

$$I = \int_{[0, \infty)^n} d^n x \prod_{i=1}^n x_i^{A_i} \prod_{k=1}^m P_k(\mathbf{x})^{B_k}, \quad (10)$$

with polynomials $P_k(\mathbf{x}) = \sum_{\alpha} c_{k,\alpha} \mathbf{x}^{\alpha}$ (real positive coefficients $c_{k,\alpha}$, possibly depending on kinematic parameters; integer exponent vectors $\alpha \in \mathbb{Z}_{\geq 0}^n$), monomial exponents $A_i \in \mathbb{C}$, and polynomial exponents $B_k \in \mathbb{C}$. Throughout, $\mathbf{x}^{\alpha} = \prod_i x_i^{\alpha_i}$. The two structural difficulties of (10) are (i) the non-compact domain, and (ii) the fact that different monomials of the P_k dominate in different regions, so no single global rescaling tames the integrand. The algorithm removes both by decomposing the domain along the tropical (Newton polytope) fan and bringing each piece to a flat $O(1)$ integrand on the unit cube, which is then sampled uniformly.

7.2 Step 0: The Tropical Fan (computed by other code)

Let $P = P_1 \cdots P_m$ and let $\text{NP}(P)$ be its Newton polytope (the convex hull of the exponent vectors of P). The *normal fan* of $\text{NP}(P)$ partitions $\mathbb{R}^n$ into polyhedral cones, one full-dimensional cone per vertex of $\text{NP}(P)$; refining each cone into simplicial cones gives the *tropical fan* used here.

This fan is **not computed by the evaluation package**. Separate code (`tropical_fan.wl`) produces it and hands over

$$\text{fanData} = \{\text{dualVertices}, \text{simplexList}\},$$

a list of integer ray vectors $\rho \in \mathbb{Z}^n$ and a list of n -tuples of ray indices, one tuple $\sigma = (\rho_1, \dots, \rho_n)$ per simplicial cone. The evaluation algorithm relies on exactly four properties:

1. **Simpliciality:** each cone has n linearly independent integer rays ($\det \neq 0$; violations are caught per sector via `TropicalEval::degenerate / ::notsimplicial`).
2. **Completeness:** the union of the cones is all of $\mathbb{R}^n$.
3. **Essential disjointness:** distinct cones overlap only on measure-zero boundaries.
4. **Vertex duality:** for each cone there is a single monomial of each P_k (the vertex of its Newton polytope dual to the cone) that attains the tropical minimum throughout the cone. This is what guarantees a constant term after the factoring of Step 2.

Any `fanData` with these properties is acceptable — in particular explicit hand-constructed fans, which the lifted tests 23A/23D use when the automatic construction is unavailable (degenerate lifted polytopes).

7.3 Step 1: Sector Decomposition by Monomial Maps

For a simplicial cone σ with rays $\{\rho_1, \dots, \rho_n\}$, define the integer matrix

$$M = -\text{Transpose}[\rho_1, \dots, \rho_n] \quad \text{i.e.} \quad M_{ij} = -(\rho_j)_i, \quad (11)$$

and the monomial change of variables

$$x_i = \prod_{j=1}^n y_j^{M_{ij}}, \quad \mathbf{y} \in (0, 1]^n. \quad (12)$$

Why this partitions the domain.

Taking logs, $\log x = M \cdot \log y$ with $\log y_j \leq 0$, so the image of $(0, 1]^n$ under (12) is exactly the set of $\mathbf{x}$ with $\log \mathbf{x} \in \text{cone}(\rho_1, \dots, \rho_n) = \sigma$ (the j -th column of $-M$ is ρ_j , and $-\log y_j \geq 0$ are the cone coordinates). Because the cones of the fan cover $\mathbb{R}^n$ with essentially disjoint interiors (Section 7.2), the images of the unit cube under the maps (12) cover $(0, \infty)^n$ in the same way:

$$I = \sum_{\sigma} I_{\sigma}, \quad I_{\sigma} = \int_{[0,1]^n} d^n y [\text{integrand} \circ (12)] \times [\text{Jacobian}]. \quad (13)$$

This is an exact, purely algebraic domain decomposition — no partition of unity, no overlap corrections.

Jacobian and raw exponents.

The Jacobian of (12) is

$$dx_1 \cdots dx_n = |\det M| \cdot \prod_j y_j^{(\sum_i M_{ij})-1} dy_1 \cdots dy_n, \quad (14)$$

and the monomial prefactor transforms as $\prod_i x_i^{A_i} = \prod_j y_j^{\sum_i A_i M_{ij}}$. Combining the two and writing the result in the convention “integrand = $\prod_j y_j^{a_j-1} \times (\cdots)$ ” (the -1 from the measure dy_j/y_j is kept separate throughout), the *raw exponents* are

$$\text{rawA}_j = \sum_i (A_i + 1) M_{ij} \quad (\text{in code: } \text{rawAVals} = (\text{monoExps} + 1) \cdot \text{mMatrix}), \quad (15)$$

so that after Step 1 the sector integral reads

$$I_\sigma = |\det M| \int_{[0,1]^n} d^n y \prod_j y_j^{\text{rawA}_j-1} \prod_k P_k(\mathbf{y}^M)^{B_k}. \quad (16)$$

Each monomial $c \mathbf{x}^\alpha$ of each P_k becomes $c \mathbf{y}^{\alpha \cdot M}$; the transformed exponent vectors $\alpha \cdot M$ generally have *negative* entries, which Step 2 removes.

7.4 Step 2: Tropical Factoring (exact)

After the M -transformation, each polynomial $P_k(\mathbf{y})$ is a sum of monomials whose exponents $e_m = \alpha_m \cdot M$ may have negative entries. Define, componentwise,

$$d_{k,j} = \min_m \{e_{m,j}\} \quad (\text{min over all monomials } m \text{ of } P_k), \quad (17)$$

and factor the minimum out:

$$P_k(\mathbf{y}) = \mathbf{y}^{d_k} \cdot Q_k(\mathbf{y}). \quad (18)$$

This identity is algebraic — *exact for any coefficient values*, however extreme. By construction every monomial of the *cleared polynomial* Q_k has non-negative exponents, and by the vertex-duality property of the fan (Section 7.2, item 4) the cone’s dominant monomial attains the minimum in *every* coordinate simultaneously, so Q_k contains a genuine constant term. Since all coefficients are positive, this gives the key estimate

$$Q_k(\mathbf{y}) \geq c_{\text{dom}} > 0 \quad \text{for all } \mathbf{y} \in [0, 1]^n : \quad (19)$$

the singular behavior of P_k near the coordinate hyperplanes has been moved entirely into the explicit monomial $\mathbf{y}^{d_k}$, and $Q_k^{B_k}$ is a smooth bounded function on the closed unit cube.

7.5 Step 3: Effective Exponents and the Convergence Gate

Absorbing the factored monomials $\mathbf{y}^{d_k}$ raised to the powers B_k into the measure exponents of Step 1 defines the *effective exponents*

$$a_{\text{eff},j} = \text{rawA}_j + \sum_k B_k d_{k,j} \in \mathbb{C}, \quad (20)$$

after which the sector integral has the fully explicit form

$$I_\sigma = |\det M| \int_{[0,1]^n} d^n y \prod_j y_j^{a_{\text{eff},j}-1} \prod_k Q_k(\mathbf{y})^{B_k}. \quad (21)$$

Because Q_k is bounded above and below on the cube, I_σ converges if and only if

$$\operatorname{Re}(a_{\text{eff},j}) > 0 \quad \text{for all } j. \quad (22)$$

This is the **convergence gate**: `ProcessSector` flags any sector violating it as `IsDivergent`, and the `v2` driver then aborts the whole evaluation with `TropicalEval::divergentinput` (this package evaluates convergent integrals only; the original `TROPICAL_MONTE_CARLO` package retains the pole-extraction machinery).

7.6 Step 4: Flattening

The integrand of (21) still contains the factors $y_j^{a_j-1}$ (writing $a_j \equiv a_{\text{eff},j}$), which are integrable but sharply peaked at $y_j = 0$ whenever $\operatorname{Re} a_j < 1$ — poison for uniform sampling. The substitution

$$y_j = (y'_j)^{1/a_j} \quad \implies \quad y_j^{a_j-1} dy_j = \frac{1}{a_j} dy'_j \quad (23)$$

absorbs them *completely* into the measure. Concretely, each monomial exponent vector e_m of each Q_k is divided componentwise by the a_j ($e_{m,j} \rightarrow e_{m,j}/a_j$, the *flattened* polynomials), and the constants collect into the sector prefactor

$$\text{prefactor}_\sigma = \frac{|\det M|}{\prod_j a_j}. \quad (24)$$

For complex a_j the substitution is the analytic continuation of the real-exponent case; only $\operatorname{Re} a_j > 0$ is needed, and the resulting complex-valued sector integrals are verified against direct numerical integration by the validation layer (Section 7.10). Geometrically, a nonzero $\operatorname{Im} a_j$ turns this flattening into a rotation of the integration contour onto a logarithmic spiral, which also suppresses the Monte Carlo variance — worked out explicitly in Section 7.18.

7.7 The Master Formula

Putting Steps 1–4 together, the algorithm rests on the exact identity

$$I = \sum_\sigma \frac{|\det M_\sigma|}{\prod_j a_j^\sigma} \int_{[0,1]^n} d^n y' \prod_k Q_k^\sigma(\mathbf{y}'^{1/a^\sigma})^{B_k} \quad (25)$$

where every quantity is produced symbolically, per sector, by `ProcessSector`. No approximation has been made at any point: the fan decomposition, the tropical factoring, and the flattening are all exact changes of variables. Each integrand in (25) is a smooth, bounded, $O(1)$ function on the unit cube — by design the ideal target for plain uniform Monte Carlo.

7.8 Step 5: Uniform Monte Carlo and Error Estimation

Each sector integral in (25) is estimated independently with N uniform samples ("`NSamples`", default 10^6 , *per sector*):

$$\hat{I}_\sigma = \frac{1}{N} \sum_{r=1}^N f_\sigma(\mathbf{y}^{(r)}), \quad \mathbf{y}^{(r)} \sim \text{Uniform}[0,1]^n, \quad (26)$$

where f_σ is the flattened integrand including its prefactor. Real and imaginary parts are accumulated separately with Welford's online algorithm (Section 7.16), giving per-sector sample variances $\widehat{\text{Var}}(\hat{I}_\sigma)$ at $O(1)$ memory. The final estimate and error are

$$\hat{I} = \sum_\sigma \hat{I}_\sigma, \quad \text{err}^2 = \sum_\sigma \widehat{\text{Var}}(\hat{I}_\sigma) = \sum_\sigma \frac{M_{2,\sigma}}{N(N-1)}, \quad (27)$$

reported separately for the real and imaginary parts (columns `ReErr` / `ImErr`). Summing variances is valid because the sector estimates use disjoint sample draws. The total cost per kinematic point is $N \times (\text{number of sectors})$ integrand evaluations, and the error scales as $1/\sqrt{N}$.

Reproducibility.

Sampling uses `std::mt19937_64` seeded deterministically per kinematic point: `seed = SeedBase + kp` (default base 42, option "`SeedBase`" of `GenerateCppMonteCarlo`). Repeated runs with the same inputs produce bit-identical results, independent of the number of OpenMP threads (threads parallelize over kinematic points, each of which owns its own RNG stream).

Caveat: heavy tails.

The error bar is a *sample* estimate. When coefficients span many orders of magnitude, the cleared polynomial Q can be far from $O(1)$ on a small region of the cube; if that region is never sampled, both the estimate and its error bar are too small. This is the failure mode addressed by the lifting module (Sections 5.2.2 and 7.11) and diagnosed by `CheckFlatteningMagnitude`.

7.9 Numerical Integrator: Monte Carlo vs. VEGAS

The tropical decomposition produces the *value*; the sampler that evaluates each flattened sector integral in (25) is a *second-order optimization* on top. The package ships two, selected by the "`Integrator`" option (default "`MC`"):

- "`MC`" — the plain uniform Monte Carlo of Section 7.8 (the historical default; unchanged).
- "`VEGAS`" — CUBA's Vegas routine (T. Hahn, *Comput. Phys. Commun.* **168** (2005) 78; <https://feynarts.de/cuba/>): adaptive importance sampling on a Sobol'low-discrepancy base. Because it returns the *same* integral, more accurately, switching integrators must not change the answer — only its error bar. That equality is asserted case by case in `EXAMPLES/test_vegas.wl`.

Why VEGAS, and the $\sqrt{\cdot}$ edge.

The sampler study in `TEST/ (REPORT.md, condensed in RESULTS.md)` benchmarks MC, randomized Sobol'QMC, and the four CUBA routines on the *post-flattening* integrand for $n = 2 \dots 8$. Flattening (Section 7.6) makes the integrand $O(1)$ but *not smooth*: a non-unit effective exponent a_i injects a $\sqrt{\cdot}$ -type cube-edge singularity (e.g. $a_i = \frac{1}{2}$ leaves a factor behaving like $\sqrt{y_i}$ near an edge). Plain MC is the weakest method on this integrand; deterministic cubature (Cuhre) only shines when flattening happens to leave integer effective exponents (a globally smooth integrand). **Vegas wins in every tested case**: its adaptive grid absorbs the $\sqrt{\cdot}$ edge structure, it pairs a QMC-like convergence rate ($\sim N^{-1.0 \dots 1.7}$ vs MC's $N^{-1/2}$) with importance sampling, and that rate is *stable in dimension* — it is the only method that scales cleanly to $n = 7, 8$. On the parity examples (`test_vegas.wl`, cases V1–V4) Vegas reproduces the MC value to within combined error while delivering 1–3 orders tighter accuracy at a *smaller* evaluation budget.

API and contract.

"VEGAS" is accepted by `GenerateCppMonteCarlo`, `EvaluateTropicalMC`, and (passed through) `EvaluateTropicalMCLifted`. Tuning options (used only for "VEGAS"): "`VegasEpsRel`" (`Automatic` $\Rightarrow 10^{-\text{PrecisionGoal}}$, clamped to $[10^{-12}, 10^{-2}]$), "`VegasSeed`" ($0 \Rightarrow$ Sobol; recommended), "`VegasNStart`", "`VegasNIncrease`", "`VegasNBatch`" (all `Automatic` by default; see the dimension note below), "`VegasMinEval`". The runtime `n_samples` argument is reused verbatim as Vegas's `maxeval per sector`. The complex integrand is first-class: each Vegas call uses `ncomp = 2` (component 0 = Re, 1 = Im), and the per-kinematic-point output contract is unchanged (`re im reErr imErr`, with sector errors summed in quadrature), so the entire result-reading path is shared with MC. "VEGAS" requires the CUBA library at compile time

(`cuba.h + libcuba`, auto-detected under `/opt/homebrew`, `/usr/local`, `/usr`); if it is absent the driver issues `TropicalEval::nocuba` and returns `$Failed` — it never silently falls back to MC. With the default "MC" the generated source, compile flags, and results are byte-identical to the pre-VEGAS pipeline (regression-gated).

Grid sizing in high dimension (important).

The per-iteration grid budget (`VegasNStart`, growing by `VegasNIncrease`) must populate Vegas's adaptive stratification in dim dimensions. The historical fixed defaults (1000/500/1000) are tuned for $n \lesssim 4$ and are *far* too small for $n \gtrsim 6$: Vegas then locks onto a coarse, wrong stratification and returns a **confidently wrong value with a deceptively tight error bar** (measured: $\sim 1\%$ off on an $n = 8$ continuous integrand, $\sim 45\%$ off on its lifted counterpart). The three sizing options therefore now default to `Automatic`: `EvaluateTropicalMC` resolves them from the maximum sector dimension d — the historical values for $d \leq 4$ (so low-dimensional results are unchanged) and $\sim 1000 \cdot 3^{d-4}$ for $d \geq 5$ (e.g. `NStart` $\approx 8 \times 10^4$ at $d = 8$). Because `maxeval` (`NSamples`) must still allow several refinement iterations *on top of* the larger grid, the driver emits `TropicalEval::vegasbudget` when `NSamples` $< 20 \cdot \text{NStart}$; for $n = 8$ a budget of a few $\times 10^6$ per sector reproduces the reference to $\lesssim 0.1\%$. Explicit integer values are always honored. As always in this regime, *cross-check against a reference rather than trusting the quoted error bar*.

Determinism and threads. With `VegasSeed = 0` each Vegas call uses Sobolpoints, so every kinematic point is deterministic and its result is independent of the thread count. The generated kp loop carries an `#pragma omp parallel for`; with `cubacores(0,0)`, `gridno = 0`, and `statefile = nullptr` the calls hold no shared grid state. On Apple clang (no OpenMP) the loop runs serially, so this machine is unaffected; on a gcc/`-fopenmp` build the 1-thread-vs- N -thread bitwise-equality check is a recommended one-time verification (CUBA is not formally documented thread-safe). The fallback, if needed, is to wrap the Vegas call in `#pragma omp critical`.

Lifted sectors: VEGAS *with* the auxiliary-variable method.

The codegen for "VEGAS" handles lifted sectors (Sections 5.2.2, 7.11) correctly — the `DomainConstraint` indicator lives inside `integrand_conv_k`, so it is integrator-agnostic — and the two combine usefully. The lifted sector carries a *hard domain-indicator cutoff* (a step discontinuity, `if (log_ypstar > 0) return 0`); Vegas's importance sampling biases slightly on that discontinuity, a bias that *shrinks with the evaluation budget*, and both samplers' *error bars are optimistic* in this heavy-tail regime (they undersample the tail). Whether Vegas or MC is more accurate depends on the coefficient regime — measured on $(1 + cx_1^2 + x_2^2 + x_1x_2^2)^{-2}$ lifted with $k=2$, 2×10^6 evals each (cases L1–L3 of `test_vegas.wl`):

coefficient c	lifted-MC rel. err	lifted-VEGAS rel. err	better
10^{-4} (small)	$\sim 1.2\%$	$\sim 0.35\%$	VEGAS
10^{-6} (small)	$\sim 13\%$ (error bar lies!)	$\sim 3\%$	VEGAS (decisively)
10^6 (large)	$\sim 0.3\%$	$\sim 1\%$	MC

For the *small*-coefficient regime — the principal reason the lift exists — **lifting + VEGAS beats lifting + MC**: VEGAS's adaptive sampling handles the post-lift heavy tail far better than uniform MC, dramatically so at 10^{-6} where plain MC is $\sim 13\%$ off with a deceptively small error bar while VEGAS lands $\sim 3\%$ and converging. For a *large* coefficient the cutoff bias dominates for VEGAS and MC edges it. In either regime, cross-check against a reference rather than trusting the quoted error bar. On the standard *continuous* (unlifted) sectors there is no discontinuity and VEGAS is unconditionally 1–3 orders tighter (cases V1–V4). `EXAMPLES/test_complex_lifted_vegas_hd.wl` (Test 26) carries the whole construction to $n = 8$ with a *tiny complex* coefficient: it evaluates the same integral with and without lifting

under VEGAS, checks both against an exact factorized reference, and confirms the lifted and unlifted values agree — the high-dimensional, complex-coefficient stress test for this combination.

Batched kinematic scans (lever, not yet shipped).

For a scan over many kinematic points sharing one polynomial *structure*, the report's headline speed-up ($\sim 30\text{--}60\times$ to 1% at $n = 8$) comes from one Vegas call *per sector* that evaluates a whole chunk of points at once via `ncomp = 2 × chunk` (cap `chunk ≤ 256` so `ncomp ≤ 512`), sharing the expensive transcendental samples across points. The current package batches over points with the existing per-point loop; the `ncomp`-batched variant is documented in `bench_batch.cpp` and `TEST/RESULTS.md §8` as a follow-up.

7.10 Validation Layer

`ValidateDecomposition[spec, fanData, kinRules, pg]` checks the master formula (25) numerically before any C++ runs: it evaluates the original integrand by direct `NIntegrate` on $[0, \infty)^n$ (precision goal $pg + 1$) and each flattened sector integral by `NIntegrate` on $[0, 1]^n$ (precision goal pg , log-exp evaluation), and compares the sector sum against the direct result, firing `TropicalEval::validate` when the relative error exceeds 10^{-pg+1} . It returns an association with keys "DirectResult", "SectorSum", "RelativeError", and "SectorResults". `ValidateLiftedDecomposition` is the sibling for lifted integrands (direct integral of the *original* spec vs. lifted sector sum, with `DroppedSectors` reporting `EmptyDomain` sectors). The driver invokes these automatically when "RunChecks" $\rightarrow$ `True` and kinematic symbols are present; for parameter-free integrands call them directly, as the test suites do.

7.11 The Lifted Algorithm: Delta Elimination in Detail

A lift rule replaces the coefficient C of monomial $\mathbf{x}^\alpha$ by z^k (z a new variable, anchor $z_0 = |C|^{1/k}$), and inserts $\int_0^\infty dz \delta(z - z_0)$, which changes nothing. The $(n+1)$ -dimensional lifted integrand is decomposed on the fan of the *lifted* polynomial — whose Newton polytope now sees the extreme coefficient as geometry — and the delta constraint is resolved analytically, per sector, by `ProcessSectorLifted`:

1. **Augmented sector data.** Run the standard `ProcessSector` steps (Steps 1–3 above) in dimension $n+1$, yielding cleared polynomials and effective exponents $a = (a_1, \dots, a_{n+1})$. Let $m = (m_1, \dots, m_{n+1})$ be the z -row of the ray matrix M , so that under the sector map $z = \prod_j y_j^{m_j}$.

2. **Pivot solve.** Select a pivot p with $m_p \neq 0$ and solve the delta constraint $\prod_j y_j^{m_j} = z_0$ for the pivot variable:

$$y_p^* = z_0^{1/m_p} \prod_{j \neq p} y_j^{-m_j/m_p}. \quad (28)$$

3. **Delta Jacobian.** Writing $\delta(z - z_0)$ in terms of y_p contributes $|\partial z / \partial y_p|^{-1} = y_p^* / (|m_p| z_0)$ at the root; combining with the measure factor $(y_p^*)^{a_p-1}$ and expanding y_p^* via (28) gives the lifted sector prefactor base and exponent shifts implemented in the code:

$$\text{prefactorBase} = \frac{|\det M|}{|m_p|} z_0^{a_p/m_p-1}, \quad \tilde{a}_j = a_j - a_p \frac{m_j}{m_p} + \sum_k B_k \tilde{d}_{k,j}. \quad (29)$$

4. **Substitute and re-clear.** Substituting (28) into each cleared polynomial maps monomial $\{c, e\}$ to $\{c z_0^{e_p/m_p}, e_j - e_p m_j/m_p\}$ ($j \neq p$); negative exponents may reappear, so the factoring of Step 2 is repeated, producing the re-cleared minima $\tilde{d}_{k,j}$ above and n -variable polynomials $\tilde{Q}_k$. Whether $\tilde{Q}$ retains a constant term is recorded as `HasConstantTerm`.

5. **Pivot admissibility and ranking.** A pivot is admissible only if every $\tilde{a}_j$ is numeric, real (within 10^{-12}), and positive. Among admissible pivots the code prefers, in order: (1) `HasConstantTerm -> True`, (2) $|m_p| = 1$, (3) the largest worst integrability margin $\min_j \tilde{a}_j$. (Constant-term preservation ranks first because sectors without one can have infinite true variance.)
6. **Domain constraint.** The lifted sector only contributes where the pivot root lies in the cube: $y_p^* \in (0, 1]$, i.e. $\log y_p^* \leq 0$. Sign analysis of $\log y_p^* = (\log z_0 - \sum_{j \neq p} m_j \log y_j) / m_p$ over $\log y_j \leq 0$ classifies each sector as: always feasible (no indicator needed), never feasible (returned as `<|"EmptyDomain" -> True|>`; the sector exactly contributes zero and is dropped, with a count reported by the driver), or mixed. In the mixed case the constraint is compiled into the integrand. Because the Monte Carlo samples the *flattened* variables $y_j = (y'_j)^{1/\tilde{a}_j}$, the indicator is expressed with coefficients $m_j/\tilde{a}_j$ (key `IndicatorCoeffs`):

$$\log y_p^* = \frac{1}{m_p} \left(\log z_0 - \sum_{j \neq p} \frac{m_j}{\tilde{a}_j} \log y'_j \right) \leq 0 \quad (30)$$

and the generated C++ integrand returns 0 whenever it fails.

7. **Flatten and proceed.** The surviving n -dimensional sector (polynomials $\tilde{Q}_k$, exponents $\tilde{a}$, prefactor base above) goes through the standard flattening of Step 4 and joins the unlifted sectors in the master formula and the Monte Carlo of Step 5.

The construction is exact end to end: the lift is an identity by the round-trip check of `LiftCoefficients` (residual coefficients $c_i = C_i/z_0^{k_i}$ preserve exactness when several monomials are lifted against one anchor), and the delta resolution is an analytic change of variables. What changes is only the *geometry*: the fan of the lifted polynomial adapts to the extreme coefficient, so the per-sector integrands are closer to $O(1)$ and the Monte Carlo variance drops. Section 7.12 carries this out explicitly on a one-dimensional integral, including a plain-vs-lifted variance comparison.

Error paths.

`liftnopivot`: no admissible pivot exists (including the case where every pivot yields some $\operatorname{Re} \tilde{a}_j \leq 0$; the message lists the per-pivot $\tilde{a}$). `liftcomplex`: candidate pivots produce complex $\tilde{a}$ — no real-valued domain indicator exists; lifting requires real polynomial exponents B_k . `liftdivergent`: safety guard if the selected pivot's flattening still reports divergence. `liftdegenerate`: the lifted Newton polytope is genuinely lower-dimensional, so no complete simplicial $(n+1)$ -fan exists — supply an explicit `"FanData"`. (The automatic path is scale-robust and no longer fails merely because the ambient dimension is ≥ 4 ; it only reaches this message for a truly degenerate polytope.) `liftfandim`: the fan handed to the driver in lifted mode is not $(n+1)$ -dimensional.

7.12 Worked Example: Why Lifting Works (Toy 0 in 1D)

To see concretely *how the tropical sampling is affected*, take the smallest possible integral with an extreme coefficient (`SANDBOX/sandbox_toy0_1d.wl`; this is Test 23D of Section 11.2):

$$I = \int_0^\infty \frac{dx}{(1 + 10^6 x)^2} = 10^{-6}, \quad n = 1, \quad A = (0), \quad P = 1 + 10^6 x, \quad B = (-2). \quad (31)$$

Because $n = 1$ every quantity below can be written out by hand, and each number quoted is the actual package output (`ProcessSector`, `ProcessSectorLifted`).

Without lifting.

The Newton polytope of P is the segment $[0, 1] \subset \mathbb{R}$; its normal fan has the two rays $\rho_{\pm} = (\pm 1)$, hence two sectors.

- **Sector** ρ_- ($M = (1)$, so $x = y$, image $x \in (0, 1]$): no transformed exponent is negative, $d = 0$, the cleared polynomial is $Q = 1 + 10^6 y$, and $a_{\text{eff}} = 1$. Flattening is trivial ($a = 1$), so the function the Monte Carlo actually samples is

$$f_-(y') = (1 + 10^6 y')^{-2}, \quad y' \sim \text{Uniform}[0, 1]. \quad (32)$$

- **Sector** ρ_+ ($M = (-1)$, so $x = 1/y$, image $x \in [1, \infty)$): $P = 1 + 10^6 y^{-1} = y^{-1}(y + 10^6)$, $d = -1$, $Q = 10^6 + y$, $a_{\text{eff}} = -1 + (-2)(-1) = 1$, and $f_+(y') = (10^6 + y')^{-2} \approx 10^{-12}$.

Sector ρ_+ is harmless: $Q = 10^6 + y'$ barely moves over the cube, so f_+ is essentially constant. Sector ρ_- is the whole problem. Its $Q = 1 + 10^6 y'$ ranges over twelve orders of magnitude on $[0, 1]$, so (32) is ≈ 1 only on the strip $y' \lesssim 10^{-6}$ and $\lesssim 10^{-12}$ everywhere else — the entire integral lives in a region of width 10^{-6} that uniform sampling almost never hits.

The decomposition is still *exact* (nothing above was approximated); it is the *sampling* that is ruined. The fan is the normal fan of the *integer* Newton polytope and is blind to the coefficient 10^6 . The two monomials of P balance where $1 = 10^6 x$, i.e. at $x = 10^{-6}$ — deep in the interior of sector ρ_- — while the only sector wall sits at the cube boundary $x = 1$. Nothing in the geometry resolves the transition.

With lifting.

Replace the extreme coefficient by an auxiliary variable, $10^6 x \rightarrow z x$, with anchor $z_0 = |10^6|^{1/k} = 10^6$ for $k = 1$. The lifted polynomial is

$$P_{\text{lift}}(x, z) = 1 + z x \quad (\text{coefficient now } 1 = O(1)), \quad I = \int_0^\infty dz \delta(z - z_0) \int_0^\infty dx (1 + zx)^{-2}. \quad (33)$$

The magnitude 10^6 no longer lives in a coefficient; it has moved into the anchor z_0 , which the δ pins. The lifted Newton polytope is degenerate (1-dimensional in 2D), so an explicit complete fan of 7 rays is supplied. `ProcessSectorLifted` returns:

Cone	rays	pivot p	prefactor	$\tilde{Q}(y')$	domain	f (exact)
1	$(1, 0), (0, 1)$	2	10^{-12}	$1 + 10^{-6} y'$	none	1.0×10^{-12}
2	$(0, 1), (-1, 1)$	1	10^{-12}	$10^{-6} + y'$	constrained	5.0×10^{-7}
3	$(-1, 1), (-1, 0)$	1	10^{-6}	$1 + y'$	none	5.0×10^{-7}
4	$(-1, 0), (-1, -1)$	—	—	—	EmptyDomain (dropped)	
5	$(-1, -1), (0, -1)$	—	—	—	EmptyDomain (dropped)	
6	$(0, -1), (1, -1)$	—	—	—	EmptyDomain (dropped)	
7	$(1, -1), (1, 0)$	—	—	—	EmptyDomain (dropped)	

Take the dominant survivor, **cone 3** (rays $(-1, 1), (-1, 0)$, so $M = \begin{pmatrix} 1 & 1 \\ -1 & 0 \end{pmatrix}$, $x = y_1 y_2$, $z = 1/y_1$). Here $P_{\text{lift}} = 1 + y_2$, the cleared $Q = 1 + y_2$, the z -row is $m = (-1, 0)$, and the only admissible pivot is $p = 1$. The delta root is the *constant* $y_1^* = z_0^{1/m_1} = z_0^{-1} = 10^{-6} \in (0, 1]$ (so the sector is feasible on the whole cube — no indicator). Eliminating y_1 leaves the one-variable cleared polynomial $\tilde{Q} = 1 + y_2$, effective exponent $\tilde{a} = 1$, and prefactor base

$$\frac{|\det M|}{|m_p|} z_0^{a_p/m_p - 1} = 1 \cdot z_0^{0/(-1) - 1} = z_0^{-1} = 10^{-6}, \quad (34)$$

so the function the Monte Carlo samples in this cone is

$$f_3(y') = 10^{-6} (1 + y')^{-2}, \quad y' \sim \text{Uniform}[0, 1]. \quad (35)$$

Compare (35) with (32). The twelve-orders-of-magnitude swing is gone: $(1 + y')^{-2}$ varies only over $[\frac{1}{4}, 1]$, and the entire extreme scale has been carried *out* of the random integrand and into the exact, deterministic prefactor $z_0^{-1} = 10^{-6}$. That is precisely what “moving the extreme coefficient into the geometry” (Section 5.2.2) buys: the sampler now sees a flat $O(1)$ function. Four of the seven cones have a constant delta-root > 1 and are dropped as `EmptyDomain`; the surviving mass is split between cone 3 and the (still mildly heavy-tailed, domain-constrained) cone 2.

The comparison.

Sampling every cone with $N = 10^6$ uniform points (seed 42), reproduced by the script `SANDBOX/compare_toy0_lift.wl`, gives:

	dominant sampled f	estimate $\pm$ std. err.	dominant σ/μ
Without lifting	$(1 + 10^6 y')^{-2}$	$9.21 \times 10^{-7} \pm 5.61 \times 10^{-7}$	≈ 610
With lifting ($k=1$)	$10^{-6}(1 + y')^{-2}$	$8.82 \times 10^{-7} \pm 1.57 \times 10^{-7}$	≈ 0.41

The per-sample standard deviation of the dominant cone falls from $\sigma/\mu \approx 610$ (unlifted, (32)) to 0.41 for the flat cone 3 ((35)) — a factor of ~ 1500 — and the total Monte Carlo error bar at fixed N shrinks by $3.6\times$, i.e. $12.7\times$ in variance. The error bar is the seed-robust figure of merit here: the unlifted 9.21×10^{-7} comes with a $\pm 56\%$ uncertainty because at this seed only a handful of the 10^6 samples land in the 10^{-6} -wide spike, whereas the lifted estimate is built from cones that are genuinely $O(1)$.

What this example also shows.

Lifting is not a panacea. Cone 2 keeps $\sigma/\mu \approx 410$ because its re-cleared polynomial retains only the *small* constant term 10^{-6} (`HasConstantTerm` is `True`, but the constant is tiny), so $\tilde{Q}^{-2}$ still spikes near the domain-constraint boundary — the heavy-tail caveat of Section 5.2.2. The degenerate 1D polytope also forces a generic ray-fan rather than one adapted to the monomial balance, which caps the gain at $\sim 13\times$ here. For a non-degenerate Newton polytope the effect is cleaner: the full-dimensional benchmark $P = 1 + 10^6 x_1^2 + x_2^2 + x_1 x_2^2$ (Test 23B, $k = 2$) reaches a $17.9\times$ variance reduction with an automatically-built fan (see `SANDBOX/benchmark_results.md`).

7.13 Worked Example, step by step: a *small* coefficient becomes an $O(1)$ flattened integrand

The example above used a *large* coefficient; the small-coefficient case is the mirror image, and is worth doing explicitly because it is the regime the lift is most often wanted in (tiny coefficients, e.g. a perturbation of a positive polynomial). Take the smallest such integral,

$$I = \int_0^\infty \frac{dx}{(1 + \varepsilon x)^2} = \frac{1}{\varepsilon}, \quad \varepsilon = 10^{-6}, \quad A = (0), \quad P = 1 + \varepsilon x, \quad B = (-2), \quad (36)$$

so $I = 10^6$. The goal is to watch the flattened integrand the Monte Carlo actually samples, and to see that *after lifting it is $O(1)$ once the coefficients are accounted for*, with the entire 10^6 scale moved out into a deterministic prefactor. Every number below is the package output.

Step 1 — without lifting, the dominant flattened integrand is not $O(1)$.

The Newton polytope is again the segment $[0, 1]$, two sectors $\rho_\pm$. Now the mass sits in ρ_+ (the region $x \in [1, \infty)$), because $(1 + \varepsilon x)^{-2}$ stays ≈ 1 all the way out to $x \sim 1/\varepsilon = 10^6$ before it decays. In that sector $M = (-1)$, $x = 1/y$, and

$$P = 1 + \frac{\varepsilon}{y} = y^{-1} \underbrace{(y + \varepsilon)}_Q, \quad d = -1, \quad a_{\text{eff}} = -1 + (-2)(-1) = 1, \quad (37)$$

so the cleared polynomial is $Q = y + \varepsilon$ — its *constant term is the tiny* $\varepsilon = 10^{-6}$. Flattening is trivial ($a_{\text{eff}} = 1$), and the sampled function is

$$f_+(y') = (y' + \varepsilon)^{-2}, \quad y' \sim \text{Uniform}[0, 1]. \quad (38)$$

This swings from $f_+(0) = \varepsilon^{-2} = 10^{12}$ down to $f_+(1) \approx 1$: it is $O(1)$ only on the strip $y' \lesssim \varepsilon$ that carries the entire integral $\int_0^1 f_+ = 10^6$, and uniform sampling almost never lands there. The measured per-sample spread is $\sigma/\mu \approx 577$. The culprit is exactly the ε sitting on the constant term of Q .

Step 2 — lift the small coefficient into the geometry.

Replace $\varepsilon x \rightarrow z x$ with anchor $z_0 = |\varepsilon|^{1/k}$. Taking $k = 1$ gives $z_0 = \varepsilon = 10^{-6}$ and

$$P_{\text{lift}}(x, z) = 1 + z x \quad (\text{coefficient now } 1 = O(1)), \quad I = \int_0^\infty dz \delta(z - z_0) \int_0^\infty dx (1 + zx)^{-2}. \quad (39)$$

This is the *same* lifted polynomial as the large-coefficient Toy 0; only the anchor differs ($z_0 = 10^{-6}$ here vs. 10^6 there). The lifted polytope is degenerate, so the same explicit 7-ray fan is used.

Step 3 — delta-eliminate and flatten the dominant cone.

Take the dominant survivor, the cone with rays $\rho_1 = (1, -1), \rho_2 = (1, 0)$, so

$$M = -[\rho_1 \ \rho_2]^\top = \begin{pmatrix} -1 & -1 \\ 1 & 0 \end{pmatrix}, \quad x = y_1^{-1} y_2^{-1}, \quad z = y_1. \quad (40)$$

Then $P_{\text{lift}} = 1 + z x = 1 + y_2^{-1} = y_2^{-1}(1 + y_2)$, so the cleared polynomial is $Q = 1 + y_2$ — **constant term 1**, $O(1)$, the small ε is gone from the polynomial. The z -row of M is $m = (1, 0)$; the only pivot is $p = 1$, and the delta constraint $z = \prod_j y_j^{m_j} = y_1 = z_0$ has the *constant* root

$$y_1^* = z_0^{1/m_1} = z_0 = 10^{-6} \in (0, 1], \quad (41)$$

so the cone is feasible on the whole cube (no indicator). Eliminating y_1 (which Q does not contain) leaves the one-variable cleared polynomial $\tilde{Q} = 1 + y_2$, effective exponent $\tilde{a} = 1$, and prefactor base

$$\frac{|\det M|}{|m_p|} z_0^{a_p/m_p-1} = 1 \cdot z_0^{0/1-1} = z_0^{-1} = 10^6. \quad (42)$$

Flattening is trivial ($\tilde{a} = 1, y_2 \rightarrow y'$), so the function the Monte Carlo samples in this cone is

$$f_*(y') = z_0^{-1} (1 + y')^{-2} = 10^6 (1 + y')^{-2} \quad y' \sim \text{Uniform}[0, 1]. \quad (43)$$

Step 4 — read off the $O(1)$ structure.

Compare (43) with the unlifted (38). The random part is $(1 + y')^{-2}$, which lives in $[\frac{1}{4}, 1]$ — genuinely $O(1)$, with measured $\sigma/\mu \approx 0.41$ (down from 577, a $\sim 1400\times$ drop in per-sample spread). Every coefficient *inside* the sampled polynomial $\tilde{Q} = 1 + y'$ is $O(1)$; the whole extreme scale $1/\varepsilon = 10^6$ has been carried, exactly, into the deterministic prefactor z_0^{-1} . That is the precise sense in which “the flattened integrand is $O(1)$ once the coefficients are accounted for”: the lift rebalances the polynomial so the part that is sampled is flat and the part that is extreme is known in closed form. The full per-cone breakdown (package output) is

Cone	rays	pivot	prefactor	sampled $f(y')$	σ/μ
4	$(-1, 0), (-1, -1)$	2	10^{-6}	$10^{-6}(1 + 10^{-12}y')^{-2}$	≈ 0
5	$(-1, -1), (0, -1)$	1	10^{-6}	$10^{-6}(y' + 10^{-12})^{-2}$ (constrained)	heavy
6	$(0, -1), (1, -1)$	1	1	$(y' + 10^{-6})^{-2}$ (constrained)	≈ 577
7	$(1, -1), (1, 0)$	1	10^6	$10^6(1 + y')^{-2}$ (43)	≈ 0.41

(cones 1–3 are dropped as `EmptyDomain`). Cone 7 is the flat, $O(1)$ piece; as in the large case, the degenerate 1D fan still leaves part of the mass in domain-constrained, heavy-tailed cones (5,6), which is why a degenerate polytope caps the gain. A *non-degenerate* lift removes even those: in the 8-dimensional, tiny complex-coefficient Test 26 every lifted sector keeps an $O(1)$ constant term (`HasConstantTerm all True`), and lifted VEGAS reaches $\sim 0.1\%$.

7.14 Choosing the Lift Anchor z_0 (the integer k)

The worked examples above pick k by hand ($k = 1$ for Toy 0, $k = 2$ for Test 23B). This section gives the principled rule that automates that choice, together with the numerical study (`AUXT/`) that validates it. The rule is now wired into `DetectExtremeCoefficients` (the “`AnchorRule`” option below), replacing the previous placeholder that always returned $k = 1$.

The anchor is not a free knob.

For the lift to be the exact identity checked by `LiftCoefficients`, the anchor must satisfy $z_0^k = |C|$, i.e.

$$z_0 = |C|^{1/k}. \quad (44)$$

So z_0 and k are *locked together*: k must be a positive integer (it is the z -exponent of the lifted monomial and a vertex coordinate of the lifted Newton polytope), and choosing that integer *determines* z_0 . The reachable anchors form the discrete ladder $z_0 \in \{|C|, |C|^{1/2}, |C|^{1/3}, \dots\} \rightarrow 1$ as $k \rightarrow \infty$. The only real question is therefore: *which integer k puts z_0 in the best place?*

Two mechanisms, pulling opposite ways.

k enters the construction twice, and keeping the two effects separate is the whole argument.

- (a) **Geometry (the gate), set by k as an integer.** The lifted vertex at auxiliary-exponent k fixes the lifted Newton polytope, its normal fan, the ray matrices and $|\det M|$, whether an admissible pivot exists, and whether the re-cleared sector keeps a constant term (`HasConstantTerm`) with positive integrability margins $\text{Re } \tilde{a}_j > 0$. None of this depends on $|C|$. *Large k hurts here*: it inflates $|\det M|$, multiplies the cone count and `EmptyDomain` drops, shrinks the margins toward 0 (heavy tails / `liftnopivot`), and can tip the polytope into degeneracy (`liftdegenerate`).
- (b) **Anchor magnitude, set by $z_0 = |C|^{1/k}$.** The deterministic prefactor absorbs $z_0^{a_p/m_p - 1}$ and carries the bulk of the extreme scale out of the random integrand (this is why Toy 0’s dominant cone becomes the flat $10^{-6}(1 + y')^{-2}$ of (35)). What is *left* inside the sampled polynomial is the residual coefficient spread $z_0^{\Delta_{e_p}/m_p}$, with Δ_{e_p} the spread of pivot-coordinate exponents. *Large k helps here*: as $z_0 \rightarrow 1$ every residual power tends to $\mathcal{O}(1)$ and every surviving sector flattens.

Mechanism (b) wants $z_0 \rightarrow 1$ (k large); mechanism (a) wants k small. Note that “is z_0 huge?” is *not* itself the problem — the large prefactor is deterministic and never enters the Monte Carlo variance; only the residual spread does.

The rule.

The principled stopping point is self-consistency with the detector. The package already declares a coefficient “extreme” when it leaves the band $[1/\tau, \tau]$ (`DetectExtremeCoefficients` threshold τ , default 10^3); by the same standard an anchor is “tame” exactly when $z_0 \in [1/\tau, \tau]$. So pick the *smallest* integer k that brings the anchor back inside the band:

$$k^* = \max\left(1, \left\lceil \frac{\lceil \log |C| \rceil}{\log \tau} \right\rceil\right) \xrightarrow{\tau=10^3} \max\left(1, \left\lceil \frac{\log_{10} |C|}{3} \right\rceil\right). \quad (45)$$

Going past k^* buys only sub-threshold residual shrinkage (the detector would no longer even flag z_0) while paying rising geometry cost — so the smallest qualifying k , not the largest feasible one, is the sweet spot. The same formula serves large *and* small coefficients because it uses $|\log |C||$. It reproduces every hand-picked k in the v2 benchmarks:

Case	$ C $	$k^* = \lceil \log_{10} C /3 \rceil$	$z_0 = C ^{1/k^*}$	hand-picked k
Case A ($1 + 10^6 x_1^2 + \dots$)	10^6	2	10^3	2
Case B primary ($\dots + 10^4 x_1^2 \dots$)	10^4	2	10^2	2
Example 20 ($10^{-4} x_1^2 \rightarrow z^2$)	10^{-4}	2	10^{-2}	2

When several monomials are lifted against one shared z_0 , fix the anchor from the primary (most extreme) coefficient via (45), then choose each secondary k_i so that $|C_i|^{1/k_i}$ lands as close as possible to that shared anchor (the residuals $c_i = C_i/z_0^{k_i}$ are then $\mathcal{O}(1)$). If two extreme coefficients cannot share an in-band anchor, prefer a single-rule lift on the dominant one (see the Case B trap below).

Numerical validation (AUXT/).

The rule was stress-tested by a k -sweep over a magnitude-laddered battery ($|C| = 10^3$ – 10^9 and 10^{-4} – 10^{-7} , plus the degenerate Case C, the multi-rule Case B, and an $n = 3, 4$ cost curve), scoring every (case, k) cell on *true* variance from NIntegrate — not the optimistic sample variance — and on accuracy against a trusted oracle. The design, theory, and auto-generated results live in AUXT/z0_numerical_testing_plan.md, AUXT/anchor_selection_procedure.md, and AUXT/z0_sweep_results.md. Three hypotheses were checked:

- **H1 (the rule).** k^* is always a gate-passing, near-optimal choice. Because the dominant surviving cone is k -invariantly flat (its σ/μ is constant down each sweep — the prefactor absorbs the anchor scale), lifting beats the unlifted baseline at *every* gate-passing k ; the only k -dependence that rises above Monte Carlo noise is accuracy. In the cases where accuracy genuinely resolves k , the optimum sits at k^* or k^*+1 — never below — so k^* is never too large.
- **H2 (gate dominance).** For the degenerate Case C, *every* swept k fails the gate (`HasConstantTerm = False`) with infinite true variance, so the harness correctly reports “do not lift.” Crucially the *finite* sample σ is k -independent there ($k=2 \equiv k=4$): ranking on it would be a false win — the geometry gate must override the anchor target.
- **H3 (monotone-then-flat).** The anchor-flattening of mechanism (b) shows up in accuracy: the relative error falls as k rises toward k^* and then plateaus, confirming there is nothing to gain past k^* .

The study also reproduces the two classic traps. The **Case B trap**: a two-rule $k=1$ lift reports a spectacular sample-variance reduction but is $\sim 92\%$ off the true value (one sector has `HasConstantTerm = False`, so its sample σ is optimistic), while the single-rule $k=2$ lift is an honest 3.2% — the reason headline reductions are computed from true, not sample, σ . The **Case C trap** is H2 above. The verdict is a go for shipping k^* as a strict, safe improvement over the old hardcoded $k = 1$, with an optional $+1$ correction at the band edge (next paragraph) flagged as a follow-up tuned on this dataset.

The band-edge correction (optional).

The one place k^* slightly under-shoots is moderate coefficients within about one decade of τ : there the ceiling in (45) rounds k down to the band edge, leaving a residual spread still > 1 , and the sweep’s accuracy optimum is k^*+1 . The “BandEdgeGuard” option adds that $+1$ when $|\log |C||$ is within one decade of $\log \tau$. It is *off by default*, because k^* alone reproduces every

hand-tuned benchmark k (table above); turning it on trades exact agreement with those hand-picks for the sweep's accuracy-optimal k at the band edge (e.g. $k=3$ rather than 2 for $|C| = 10^4$ or 10^{-4}).

How it is automated.

`DetectExtremeCoefficients` now computes `SuggestedK` from (45) via its "AnchorRule" option (default "kStar"; "Unit" restores the legacy `SuggestedK` $\rightarrow$ 1; a function `f[mag,threshold]` supplies a custom rule), with the optional "BandEdgeGuard". Both options pass through `EvaluateTropicalMCLifted`, so "LiftRules" $\rightarrow$ Automatic now lifts at k^* rather than $k = 1$. Decision order in practice: (1) the geometry gate of mechanism (a) comes first — if no k yields a surviving sector with `HasConstantTerm` and positive margins, *do not lift*; (2) among gate-passing k , take the smallest with z_0 in band, i.e. k^* ; (3) tie-break toward simpler geometry (a $|m_p| = 1$ pivot and the largest worst-case margin), exactly the ranking `ProcessSectorLifted` already uses.

7.15 Log-Exp Polynomial Evaluation

In the C++ code, monomials with fractional or complex exponents are evaluated using:

$$c \cdot y_1^{\alpha_1} \cdots y_n^{\alpha_n} = c \cdot \exp(\alpha_1 \log y_1 + \cdots + \alpha_n \log y_n) \quad (46)$$

This is numerically stable because:

- For $y_i \in (0, 1]$: $\log(y_i) \in (-\infty, 0]$, smooth and monotone.
- As $y_i \rightarrow 0^+$: $\log(y_i) \rightarrow -\infty$, but $\exp(\cdots) \rightarrow 0$ gracefully.
- No branch cuts or sign issues for real $y_i > 0$.
- Works for complex α (the result is complex-valued).
- Avoids platform-dependent behavior of `pow(y, fractional)`.

7.16 Welford Online Variance Estimation

For each integrand function and kinematic point, Monte Carlo estimates the mean and variance using Welford's online algorithm:

```
mean = 0, M2 = 0
for k = 1, 2, ..., N:
    v = integrand(random_point)
    delta = v - mean
    mean += delta / k
    delta2 = v - mean
    M2 += delta * delta2

variance = M2 / (N - 1)
std_error = sqrt(variance / N)
```

Properties:

- Numerically stable (no catastrophic cancellation).
- $\mathcal{O}(1)$ memory per integrand (just mean, M2, count).
- Real and imaginary parts tracked independently.
- Sector sums are independent: $\text{Var}(\text{total}) = \sum \text{Var}(\text{sector})$.

7.17 Treatment of Complex Exponents

The exponents A_i (monomial) and B_j (polynomial) may be complex-valued. The pipeline handles this by cleanly separating the combinatorial steps—which depend only on real integer data—from the analytic steps, which carry the full complex values throughout.

What is real/combinatorial.

The following operations depend only on the Newton polytope (the set of exponent vectors appearing in the polynomials) and are therefore insensitive to the complex parts of A_i and B_j :

- Newton polytope construction and convex hull computation.
- Normal fan and simplicial triangulation (ray vectors ρ_i are integer).
- The ray matrix $M = -\text{Transpose}[\rho_1, \dots, \rho_n]$ and its determinant.
- Tropical factoring: the minimum exponents $d_{k,j} = \min_m \{e_{m,j}\}$ are determined by the transformed polynomial exponent vectors, which are integer linear combinations of the original (integer) exponent vectors. These do not depend on A_i or B_j at all.

What is complex.

The following quantities use the full complex values of A_i and B_j , and are computed symbolically in Mathematica before being passed to C++:

- **Effective exponents:**

$$a_{\text{eff},i} = \text{raw}A_i + \sum_k B_k \cdot d_{k,i} \in \mathbb{C} \quad (47)$$

Here $\text{raw}A_i = \sum_k (A_k + 1)M_{ki}$ as in Eq. (15) (the -1 of the measure convention is kept separate), which is complex when the A_k are. The B_k contribute their complex parts through the $B_k \cdot d_{k,i}$ term.

- **Flattening:** the exponent division $e_{m,i}/a_{\text{eff},i}$ uses the full complex $a_{\text{eff},i}$, producing complex-valued flattened exponents. In the C++ code these appear inside the log-exp evaluation:

$$y_i^{e/a_{\text{eff},i}} = \exp((e/a_{\text{eff},i}) \cdot \log y_i) \quad (48)$$

which is well-defined for $y_i \in (0, 1]$ and complex exponent ratio.

- **Prefactor:**

$$\text{prefactor} = \frac{|\det(M)|}{\prod_i a_{\text{eff},i}} \in \mathbb{C} \quad (49)$$

- **Convergence check:** only $\text{Re}(a_{\text{eff},i})$ is tested. A sector is convergent if and only if $\text{Re}(a_{\text{eff},i}) > 0$ for all i . The imaginary part does not affect convergence.

Worked demonstrations: Example 2 (complex B), Example 15 (two polynomials with two different complex exponents), Example 16 (complex A_i and complex B), Examples 17 and 19 (complex exponents combined with kinematic scans and small coefficients).

Summary.

The computational strategy is:

1. Perform the tropical decomposition using only the combinatorial structure of the polynomials (integer exponent vectors). This is purely real.
2. Compute $a_{\text{eff},i}$ and the flattening transform using the full complex A_i and B_j . All downstream quantities (flattened exponents, prefactors, C++ integrands) are complex-valued.
3. In C++, all arithmetic is `std::complex<double>`. The log-exp evaluation $\exp(\alpha \log y)$ with complex α and real $y > 0$ is numerically stable and branch-cut free.

7.18 Worked Example: Complex Flattening as a Contour Rotation

The flattening of Step 4 (Section 7.6), $y_j = (y'_j)^{1/a_j}$, was introduced to absorb the integrable but sharply peaked factor $y_j^{a_j-1}$ into the measure. When the effective exponent a_j is *complex* the same substitution does something geometrically sharper: it **rotates the integration contour onto a logarithmic spiral**, and in doing so removes an oscillation that would otherwise wreck the Monte Carlo. This is worth seeing explicitly because, unlike lifting, there is no “do-nothing” alternative to compare against — the flattening is intrinsic, and the rotation is simply what it *is* for complex a_j . Take (SANDBOX/verify_complex_flatten.wl)

$$I = \int_0^\infty (1+x)^B dx = \frac{-1}{B+1}, \quad B = -2 + ib, \quad b = 6, \quad I = \frac{1}{1-ib}. \quad (50)$$

The Newton polytope is again $[0, 1]$, with the two rays $\rho_\pm = (\pm 1)$. The small- x sector ρ_- has $a_{\text{eff}} = 1$ (real) and is unremarkable. The large- x sector ρ_+ ($M = (-1)$, $x = 1/y$, $Q = 1 + y$, $d = -1$) carries the complex exponent:

$$a_{\text{eff}} = \text{raw}A + Bd = -1 + B(-1) = -1 - B = 1 - ib, \quad (51)$$

so the sector integrand on the real axis (before flattening) is

$$g(y) = y^{a-1} (1+y)^B = \underbrace{y^{-ib}}_{\text{pure phase}} (1+y)^{-2+ib}, \quad y \in (0, 1]. \quad (52)$$

Its *modulus* is perfectly tame, $|g(y)| = (1+y)^{-2} \in [\frac{1}{4}, 1]$; the trouble is entirely in the *phase*. Since $y^{-ib} = e^{-ib \ln y}$, the argument of g winds as $-b \ln y \rightarrow +\infty$ when $y \rightarrow 0$: $\text{Re } g$ and $\text{Im } g$ each oscillate in sign with a frequency that *diverges* at the origin (Figure 1B, grey). Sampling either part with uniform points is then a disaster of cancellation — the integral is small because of near-perfect sign cancellation, while the per-sample spread stays $O(1)$.

The rotation.

Flattening substitutes $y = (y')^{1/a}$. Writing $s = -\ln y' \geq 0$ and $1/a = (1+ib)/(1+b^2)$,

$$y = \exp\left(\frac{1}{a} \ln y'\right) = \underbrace{\exp\left(-\frac{s}{1+b^2}\right)}_{\text{modulus } \downarrow} \underbrace{\exp\left(-i \frac{bs}{1+b^2}\right)}_{\text{phase winds}}, \quad (53)$$

which is exactly a logarithmic spiral: as y' runs from 1 down to 0 the image point leaves $y = 1$ and spirals into the origin (Figure 1A). The imaginary part of a is the winding rate. The whole oscillating factor y^{a-1} collapses into the constant prefactor, $y^{a-1} dy = \frac{1}{a} dy'$, so what the Monte Carlo actually samples is

$$f(y') = \frac{1}{a} (1 + (y')^{1/a})^B, \quad (54)$$

a *smooth, non-oscillatory* function of y' (Figure 1B, red): along the spiral $(y')^{1/a}$ moves smoothly from 1 to 0, $1 + (y')^{1/a}$ never approaches the branch point at 0, and the phase that used to wind is gone.

Why this is legal, and what it buys.

The move is not an approximation: $y = (y')^{1/a}$ is an analytic change of variables, equivalently (Cauchy’s theorem) a deformation of the real contour onto the spiral — the integrand $y^{a-1}(1+y)^B$ is holomorphic in the swept region, its only branch points being $y = -1$ (far from the spiral, which stays in $|y| \leq 1$) and the shared, integrable endpoint $y = 0$ (where $\text{Re } a = 1 > 0$). It is precisely the analytic continuation in a of the real-exponent flattening (Section 7.17): for real $a > 0$ the “spiral” is just the real segment reparametrised; a nonzero $\text{Im } a$ tilts it

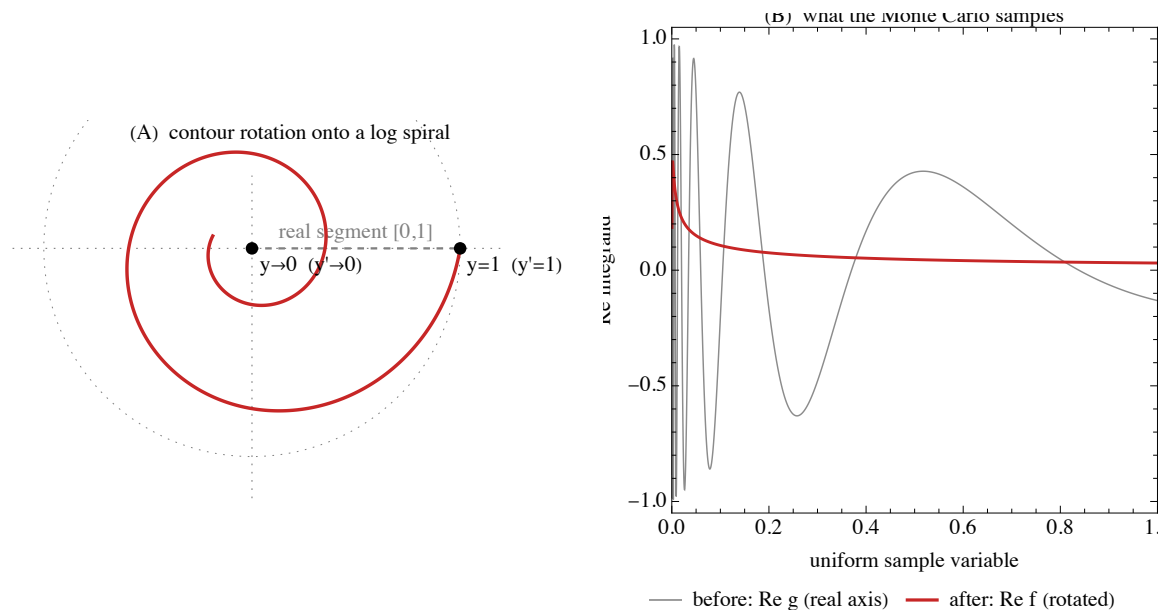


Figure 1: Complex flattening for sector ρ_+ of (50) ($a = 1 - 6i$). **(A)** The substitution $y = (y')^{1/a}$ maps the real segment $[0, 1]$ (dashed) onto a logarithmic spiral (red) running from $y = 1$ into $y = 0$; the dotted curve is the unit circle. **(B)** The real part of the integrand as a function of the uniform sample variable: before the rotation (52) it oscillates with diverging frequency near the origin (grey); after the rotation (54) it is smooth (red). The two contours give the *same* integral, by analytic continuation.

off the axis. Numerically both contours return the identical sector value $0.0619 - 0.0541i$, and the reassembled total matches $1/(1 - 6i)$ to machine precision. The payoff is variance: over 2×10^5 uniform samples the per-sample variance of the sampled integrand falls from $(\text{Var Re}, \text{Var Im}) \approx (0.138, 0.148)$ on the real contour to $(0.0059, 0.0084)$ on the rotated one — about a $20\times$ reduction, growing with b as the un-rotated oscillation gets faster. Lifting flattens a swing in *magnitude* (Section 7.12); complex flattening flattens a swing in *phase*. Both turn a function that uniform sampling cannot resolve into a flat $O(1)$ one, and both are exact.

8 Dependencies

8.1 External Software

Polymake (<https://polymake.org>)

Polyhedral-computation tool, used *only* by the fan-computation code `tropical_fan.wl` (a black box from this manual's perspective, Section 7.2). Must be installed and accessible on PATH, or in one of: `/opt/homebrew`, `/usr/local`, `/usr` (override via `$TropicalFanPolymakePath`). Can be skipped entirely with `$SkipPolymakeLoad = True` when explicit `fanData` is supplied. **Installation guide: Section 8.2.**

g++ (GCC C++ compiler)

C++17 standard required (`-std=c++17`). OpenMP support recommended (`-fopenmp`) for parallel kinematic evaluation. If OpenMP linking fails (common on macOS with clang), the code auto-retries without `-fopenmp`.

Wolfram Mathematica (version 12.0+)

The symbolic computation engine. Required for running all `.wl` files. `wolframscript`

used for command-line execution.

CUBA (optional) (<https://feynarts.de/cuba/>)

Multidimensional integration library (Vegas, Suave, Divonne, Cuhre). Used *only* by the independent cross-check examples in `EXAMPLES/tropical_eval_examples3.wl`, which integrate the original (un-decomposed) integrand directly and compare against the tropical pipeline. Install via Homebrew (`brew install cuba`) or build from source. The examples probe `/opt/homebrew`, `/usr/local`, and `/usr` for `cuba.h + libcuba` and skip gracefully with a message if not found.

8.2 Installing Polymake

Polymake is the only non-trivial dependency to install, and it is needed *only* to compute fans (Section 7.2); if you supply `fanData` explicitly and set `$SkipPolymakeLoad = True`, you can skip this section entirely. The package drives Polymake through its command-line executable (it shells out to `polymake` with a short Perl script that uses the `polytope` application), so a full working `polymake` binary on disk is required — a library-only build is not enough.

macOS (recommended: Homebrew).

```
brew install polymake
```

This installs the binary at `/opt/homebrew/bin/polymake` on Apple-silicon Macs (or `/usr/local/bin/polymake` on Intel Macs), both of which are auto-detected (see *How the package finds Polymake*, below). The formula pulls in the required Perl, GMP, MPFR, FLINT, PPL, and boost dependencies automatically. The first `polymake` invocation rebuilds its internal caches and can take a minute or two; subsequent calls are fast. This manual was validated against **Polymake 4.15** installed this way.

Linux (distribution packages).

```
# Debian / Ubuntu
sudo apt-get install polymake

# Fedora
sudo dnf install polymake

# Arch (AUR)
yay -S polymake
```

These place the binary at `/usr/bin/polymake` or `/usr/local/bin/polymake`, both auto-detected. Distribution packages are sometimes older than the upstream release; any version in the 3.x–4.x range exposing the `polytope` application’s `VERTICES` property works.

Building from source (clusters, or for the latest release).

When no package is available — e.g. on an HPC login node without root — build from the official tarball:

```
# Prerequisites: a C++14 compiler, Perl >= 5.16, GMP, MPFR,
# libxml2, boost, ninja, and (optionally) FLINT/PPL.
wget https://polymake.org/lib/exe/fetch.php/download/polymake-4.15.tar.bz2
tar xjf polymake-4.15.tar.bz2 && cd polymake-4.15
./configure --prefix=$HOME/polymake # pick any writable prefix
ninja -C build/Opt
```

```
ninja -C build/0pt install
```

Add the install prefix to your PATH (`export PATH=$HOME/polymake/bin:$PATH`) or point the package straight at the binary (see *How the package finds Polymake*, below). Full upstream instructions: <https://polymake.org/doku.php/install/install>.

Verifying the installation.

```
polymake --version
# polymake version 4.15 ...

# Confirm the polytope application + VERTICES work end to end:
polymake "use application 'polytope';
         my $c = cube(2);
         print $c->VERTICES;"
```

The second command should print the four vertices of the unit square in homogeneous coordinates. If both succeed, the package will drive Polymake correctly.

How the package finds Polymake.

On load, the variable `$TropicalFanPolymakePath` is set by probing the following locations in order, taking the first that exists:

```
/opt/homebrew/bin/polymake # Homebrew, Apple silicon
/usr/local/bin/polymake   # Homebrew (Intel) / source default
/usr/bin/polymake         # Linux distribution packages
polymake                  # fallback: resolved via $PATH
```

For a non-standard location, set the path *before* loading the package:

```
$TropicalFanPolymakePath = "/home/me/polymake/bin/polymake";
Get["tropical_eval.wl"];
```

A failed Polymake call raises `TropicalFan::polymake` with the captured `stderr`; the most common causes are a binary not on the probed paths (set `$TropicalFanPolymakePath`) and a first-run cache rebuild that exceeded the process timeout (re-run once warmed up).

8.3 Internal Dependencies

- `tropical_fan.wl` must be loaded before `tropical_eval.wl`; the latter Gets it internally, unless `$SkipPolymakeLoad` is set to `True` first.
- No external Wolfram packages required beyond the core installation.

9 Usage and Workflow

9.1 Minimal Convergent Integral

```
(* Load the package *)
Get["tropical_eval.wl"];

(* Define the integrand *)
poly = 1 + 2*x[1]^2 + x[2]^2 + x[1]*x[2]^2;
```

```

vars = {x[1], x[2]};

spec = <|
  "Polynomials"      -> {poly},
  "MonomialExponents" -> {0, 0},
  "PolynomialExponents" -> {-2},
  "Variables"        -> vars,
  "KinematicSymbols" -> {},
  "RegulatorSymbol"  -> None
|>;

(* Compute the fan *)
verts = PolytopeVertices[poly^(-1), vars];
fanData = ComputeDecomposition[verts];

(* Validate against NIntegrate *)
vr = ValidateDecomposition[spec, fanData, {}, 3];
Print["Relative error: ", vr["RelativeError"]];

(* Full MC evaluation *)
results = EvaluateTropicalMC[spec, fanData, {}];

```

9.2 Kinematic Scan (Many Evaluation Points)

```

Get["tropical_eval.wl"];

(* Polynomial with kinematic-dependent coefficients *)
poly = c0 + c1*x[1]^2 + c2*x[2]^2;

spec = <|
  "Polynomials"      -> {poly},
  "MonomialExponents" -> {0, 0},
  "PolynomialExponents" -> {-2},
  "Variables"        -> {x[1], x[2]},
  "KinematicSymbols" -> {c0, c1, c2},
  "RegulatorSymbol"  -> None
|>;

(* Fan uses unit coefficients -- structure only *)
proxyPoly = 1 + x[1]^2 + x[2]^2;
verts = PolytopeVertices[proxyPoly^(-1), vars];
fanData = ComputeDecomposition[verts];

(* Define 7200 kinematic points *)
kinPoints = Table[{1.0, 0.5+0.01*i, 1.0+0.005*i}, {i, 7200}];

(* Evaluate: one compilation, parallel execution *)
results = EvaluateTropicalMC[spec, fanData, kinPoints];

```

Note: The fan decomposition depends only on the Newton polytope structure (which monomials appear), not on numerical coefficients. The same decomposition works for all kinematic points, requiring only one C++ compilation.

9.3 Extreme Coefficients via Lifting

```

Get["tropical_eval.wl"];

(* Small coefficient 10^-4: decomposition is exact, but plain
   uniform MC is heavy-tailed. Lift the coefficient instead. *)
spec = <|
  "Polynomials"      -> {1 + 10^-4 x[1]^2 + x[2]^2 + x[1] x[2]^2},
  "MonomialExponents" -> {0, 0},
  "PolynomialExponents" -> {-2},
  "Variables"        -> {x[1], x[2]},
  "KinematicSymbols" -> {},
  "RegulatorSymbol"  -> None
|>;

(* Which coefficients are extreme? SuggestedK is the auto-chosen anchor
   exponent (kStar rule, Section "Choosing the Lift Anchor z0"); here k = 2. *)
DetectExtremeCoefficients[spec, 1000]

(* Simplest path: let the kStar rule pick k (= 2 here) automatically. *)
result = EvaluateTropicalMCLifted[spec, {}], "LiftRules" -> Automatic,
  "NSamples" -> 10^6];

(* Equivalent explicit form -- lift 10^-4 x1^2 -> z^2 x1^2, anchor z0 = 1/100.
   Use this to override the rule, or "AnchorRule" -> "Unit" for the legacy
   k = 1, or "BandEdgeGuard" -> True for the +1 band-edge correction. *)
result = EvaluateTropicalMCLifted[spec, {}],
  "LiftRules" -> {<|"PolyIndex" -> 1,
                  "ExponentVector" -> {2, 0}, "k" -> 2|>},
  "NSamples" -> 10^6];

result["Results"][[1]] (* <|"Re" -> ..., "ReErr" -> ..., ...|> *)

```

See Example 20 in `EXAMPLES/tropical_eval_examples2.wl` for the full worked version, including the plain-vs-lifted variance comparison and the exactness check via `ValidateLiftedDecomposition`.

9.4 Command-Line Execution

```

# Quick smoke test (Example 1 only)
wolframscript -file EXAMPLES/test_quick.wl

# C++ code generation and execution test
wolframscript -file EXAMPLES/test_cpp.wl

# Examples 1-14 (general pipeline)
wolframscript -file EXAMPLES/tropical_eval_examples.wl

# Examples 1-14 followed by the 6-test RunAllTests validation suite
wolframscript -file EXAMPLES/run_validation_suite.wl

# Examples 15-20 (complex exponents, small coefficients, lifting)
wolframscript -file EXAMPLES/tropical_eval_examples2.wl

# Examples 21-23 (independent cross-checks: exact Gamma + CUBA)
wolframscript -file EXAMPLES/tropical_eval_examples3.wl

# Examples 24-27 (lifting with complex, small-magnitude coefficients)

```

```
wolframscript -file EXAMPLES/tropical_eval_examples4.wl

# SeedBase runtime-argv override (re-run one binary under several seeds)
wolframscript -file EXAMPLES/test_seedbase.wl

# Lifted-pipeline test suite (Test 23)
wolframscript -file EXAMPLES/test_lifted.wl

# Complex-coefficient lifted test suite (Test 24)
wolframscript -file EXAMPLES/test_complex_lifted.wl

# CUBA cross-check of ALL test-suite integrals (requires CUBA)
wolframscript -file EXAMPLES/test_cuba.wl

# MC vs VEGAS parity demonstration (requires CUBA)
wolframscript -file EXAMPLES/test_vegas.wl

# Fan computation examples
wolframscript -file EXAMPLES/tropical_fan_examples.wl
```

10 Examples Directory

The `EXAMPLES/` directory contains worked examples and validation tests.

`tropical_eval_examples.wl` — Examples 1–14

Example	Description
1	Basic 2D convergent integral
2	Complex exponents (B_j with imaginary part)
3	Kinematic-dependent integral (coefficients are parameters)
4	Tropical factoring step-by-step walkthrough
5	MmaToC expression converter demonstration
6	C++ code generation and execution (full pipeline)
7	C++ with kinematic scan (multiple evaluation points)
8	ParsePolynomial utility function
9	Multiple polynomials (2 factors in denominator)
10	Non-trivial numerator (1 polynomial, positive B)
11	Non-trivial numerator (2 polynomials)
12	Numerator with kinematic-dependent coefficients
13	3D convergent integral (6 rays, 8 sectors)
14	4D convergent integral (8 rays, 14 sectors)

`tropical_eval_examples2.wl` — Examples 15–20: Complex Exponents and Small Coefficients

All six examples are convergent and cross-checked against `NIntegrate`.

Example	Description
15	Two polynomials, both raised to complex exponents $(1+x_1+x_2)^{-(3/2+i/2)} (1+x_1x_2)^{-(1-i/3)}$
16	Complex monomial exponents and complex polynomial exponent $x_1^{1/2+i/3} x_2^{-1/4-i/5} P^{-(2+i)}$
17	Full C++ MC via <code>EvaluateTropicalMC</code> with complex exponent $P_\lambda^{-(2+i/2)}$, kinematic scan over λ , MC vs. <code>NIntegrate</code>
18	Small coefficients (10^{-4} , 10^{-3}): exactness + variance diagnostics via <code>DetectExtremeCoefficients</code> / <code>CheckFlatteningMagnitude</code>
19	Small coefficients <i>and</i> complex exponent combined $(1 + 10^{-3}x_1^2 + x_2^2 + 10^{-2}x_1x_2^2)^{-(2+i/2)}$
20	Lifting a small coefficient ($10^{-4} \rightarrow z^2$, $z_0 = 1/100$): exactness via <code>ValidateLiftedDecomposition</code> , then plain-vs-lifted MC comparison (heavy tail vs. controlled variance); Step 7 demonstrates the automatic anchor (" <code>LiftRules</code> " $\rightarrow$ <code>Automatic</code> , the k^* rule)

tropical_eval_examples3.wl — Examples 21–23: Independent Cross-Checks

These examples validate the pipeline against methods that share no code, algorithm, or CAS with it (or with `NIntegrate`): an exact Gamma-function closed form, and the **CUBA** library integrating the *original* integrand directly on $[0, \infty)^n$ via the compactification $x_i = t_i/(1 - t_i)$. The generator that turns any numeric `IntegrandSpec` into a self-contained two-component (Re, Im) CUBA program lives in the shared helper file `cuba_common.wl` (`generateCubaSource` / `runCubaCheck`); it is also used by `test_cuba.wl`, which cross-checks every test-suite integral (Section 11.4). CUBA is an optional dependency (Section 8.1); the CUBA parts skip gracefully if it is not installed.

Example	Description
21	Exact Beta/Gamma closed form, complex exponents: $\int x_1^{a_1-1} x_2^{a_2-1} (1+x_1+x_2)^{-b} = \frac{\Gamma(a_1)\Gamma(a_2)\Gamma(b-a_1-a_2)}{\Gamma(b)}$ $(a_1 = \frac{5}{4} + \frac{i}{3}, a_2 = \frac{3}{2} - \frac{i}{5}, b = 5 + \frac{i}{2})$; exact vs. <code>NIntegrate</code> vs. tropical MC vs. CUBA Cuhre (agreement to 3×10^{-8})
22	Small coefficient 10^{-4} (Example 20's integrand): CUBA Cuhre (deterministic) and Vegas (importance-sampling MC) vs. lifted tropical MC — three unrelated algorithms agree on ≈ 13.0901
23	Complex exponent + small coefficients (Example 19's integrand): CUBA Cuhre two-component reference vs. unlifted tropical MC (honest but large error bars; lifting unavailable for complex B)

tropical_eval_examples4.wl — Examples 24–27: Lifting with Complex, Small-Magnitude Coefficients

Examples 2 and 15–19 made the integrand complex through the *exponents*; Example 20 lifted a small but *real* coefficient. This file combines the two regimes: a polynomial coefficient that is itself *complex* and of very small magnitude, fed through the lifting module. The complex coefficient is split into a real anchor $z_0 = |C|^{1/k}$ and an $\mathcal{O}(1)$ phase residual $c = C/z_0^k$ (Section 5.2.2, “Complex coefficients”), so the lifted decomposition stays exact and the integral is correctly complex-valued. All four are cross-checked against direct `NIntegrate` (Example 24 additionally runs the full C++ MC, requiring `g++`).

Example	Description
24	Single small complex coefficient $(1+i)10^{-4}$, $k = 2$: detection, lift, identity, lifted-decomposition exactness (Re and Im), and plain-vs-lifted C++ MC — plain MC is wildly heavy-tailed, lifted MC tracks the complex reference $\approx 11.63 - 2.58i$ in both components
25	Very small $(1+i)10^{-7}$ with the <i>automatic</i> anchor: the k^* rule grows the anchor exponent to $k = 3$ as $ C $ shrinks; decomposition exact
26	Two small complex coefficients lifted against one anchor (multi-rule): residual phases stay $\mathcal{O}(1)$; the round-trip is exact for both at once
27	Purely imaginary coefficient $i10^{-4}$ (residual = i), plus the boundary case: a complex <i>exponent</i> $B = -(2+i/2)$ correctly fires <code>liftcomplex</code>

Other Example Files

`tropical_fan_examples.wl` — Fan decomposition examples for various polytopes.

`test_quick.wl` — Minimal test running only Example 1.

`test_cpp.wl` — Tests C++ code generation, compilation, and execution.

`run_validation_suite.wl` — Runs Examples 1–14 (by loading `tropical_eval_examples.wl`) and then invokes `RunAllTests[]`, the 6-test validation suite (Section 11.1). The examples file *defines* `RunAllTests[]` but never calls it, so this runner is the way to exercise the suite from the command line; it exits non-zero if any test fails.

`test_seedbase.wl` — Tests the `SeedBase` runtime-argv override: compiles one binary, then re-runs it under several seeds, asserting determinism (same seed $\Rightarrow$ identical output), that omitting `argv[5]` equals passing the compile-time default, distinct MC streams across seeds, and each stream within 5σ of $\pi/8$.

`test_lifted.wl` — Test 23: the lifted-integral pipeline (four subtests, see Section 11.2).

`test_complex_lifted.wl` — Test 24: lifting with *complex*, small-magnitude coefficients (four subtests, see Section 11.3). The automated PASS/FAIL companion to Examples 24–27; exits non-zero on any failure.

`cuba_common.wl` — Shared CUBA cross-check generator (used by `tropical_eval_examples3.wl` and `test_cuba.wl`).

`test_cuba.wl` — CUBA cross-check of all test-suite integrals (Section 11.4).

`test_vegas.wl` — MC vs. VEGAS parity demonstration: runs each spec through the pipeline with `Integrator -> "MC"` and `"VEGAS"` and asserts they return the same value within combined error, with VEGAS 1–3 orders tighter (Section 7.9; requires CUBA).

`prompt_tropical_mc.txt` — Original development specification.

11 Validation and Testing

11.1 6-Test Suite (`RunAllTests[]`)

`RunAllTests[]` is *defined* in `tropical_eval_examples.wl` but not called there; run it from the command line via `wolframscript -file EXAMPLES/run_validation_suite.wl`, which first executes Examples 1–14 and then invokes the suite (exiting non-zero on any failure). The v2

suite covers convergent integrals only (the divergent and IBP tests, formerly Tests 3–4 and 8–22, were removed along with the divergence machinery):

- Test 1:** Convergent 2D, real exponents ($A = 2, 3$). Sector sum vs. direct `NIntegrate`.
- Test 2:** Convergent 2D, complex exponent ($A = 2 + 0.5i$). Tests log-exp evaluation with a complex-valued integrand.
- Test 3v2:** Negative tests. Part A: an ε -regulated spec must fail with `noregulator`. Part B: a regulator-free but divergent spec must fail with `divergentinput`.
- Test 5:** 100-point kinematic scan with complex exponent; sector decomposition verified at 5 representative points.
- Test 6:** Large coefficient range (10^{-4} to 10^8 , three cases). The tropically dominant monomial need not be the numerically largest one; the factoring is exact regardless.
- Test 7:** 3D and 4D integrals. 3D: 6 rays, 8 sectors; 4D: 8 rays, 14 sectors.

11.2 Test 23: Lifted Pipeline (`test_lifted.wl`)

- 23A (exactness):** Toy 1, $P = 1 + x_1 + 10^{-6}x_2$, $B = -3$, exact value 500000. Uses an explicit fan (the lifted polytope is degenerate). PASS: relative error $< 0.5\%$ and exactly 3 `EmptyDomain` sectors dropped.
- 23B (end-to-end C++):** $P = 1 + 10^6x_1^2 + x_2^2 + x_1x_2^2$, $B = -2$, lifted with $k = 2$ via `EvaluateTropicalMCLifted`. PASS: within 1% of `NIntegrate` and within 4σ of the MC error bar.
- 23C (error paths):** (i) complex polynomial exponent must fire `liftcomplex`; (ii) a hand-crafted sector where every pivot yields a non-positive $\tilde{a}$ component must fire `liftnopivot`.
- 23D (EmptyDomain drops):** Toy 0, $P = 1 + 10^6x_1$, $B = -2$, $k = 1$ ($z_0 = 10^6$). PASS: dropped sectors > 0 and relative error $< 0.1\%$.

11.3 Test 24: Lifting with Complex, Small-Magnitude Coefficients (`test_complex_lifted.wl`)

The automated PASS/FAIL companion to Examples 24–27 (Section 10), confirming that the auxiliary-variable module is correct when a polynomial carries a *complex* coefficient of very small magnitude. The script exits non-zero on any failure.

- 24A (single complex coeff, full C++):** $P = 1 + (1+i)10^{-4}x_1^2 + x_2^2 + x_1x_2^2$, $B = -2$, $k = 2$. PASS gates: `DetectExtremeCoefficients` flags $|C| = \sqrt{2}10^{-4}$ with $k^* = 2$; the lift identity holds and the residual has unit modulus; `ValidateLiftedDecomposition` is exact ($\sim 10^{-5}$) in both Re and Im; the C++ MC lands within 5σ in both components; and the generated source emits a `cx(...)` literal with no broken `CForm` token.
- 24B (very small, automatic k^*):** $(1+i)10^{-7}$; the `kStar` rule returns $k = 3$ and the lifted decomposition is exact ($\sim 10^{-7}$).
- 24C (multi-rule):** two small complex coefficients $(1+i)10^{-4}$ and $(1-2i)10^{-4}$ lifted against one anchor; residual magnitudes stay $\mathcal{O}(1)$, the round-trip identity holds, and the decomposition is exact ($\sim 10^{-6}$).
- 24D (boundary):** a complex polynomial *exponent* $B = -(2+i/2)$ (real coefficient) must still fire `liftcomplex` — complex coefficients are liftable, complex exponents are not.

Measured at the time of writing: all four parts PASS. Sector sums agree with direct `NIntegrate` of the complex integrand to 6×10^{-7} – 7×10^{-6} ; the C++ lifted MC for 24A reproduces $11.63 - 2.58i$ within its error bars in both components.

11.4 Independent Cross-Checks: Exact Closed Form and CUBA (Examples 21–23)

Beyond the `NIntegrate`-based validation used throughout, `tropical_eval_examples3.wl` (in `EXAMPLES/`) checks the pipeline against fully independent references (measured values at the time of writing):

- **Exact Gamma closed form** (Example 21): the generalized Beta integral with complex exponents $a_1 = 5/4 + i/3$, $a_2 = 3/2 - i/5$, $b = 5 + i/2$. Agreement: CUBA Cuhre 3×10^{-8} relative; `NIntegrate` 2×10^{-6} ; tropical sector sum 9×10^{-6} ; tropical MC within its $\sim 10^{-4}$ error bars.
- **CUBA Cuhre** (deterministic globally adaptive cubature) on the direct, compactified integrand: reproduces the small-coefficient reference 13.09008897 and the complex value $18.4048601 - 8.4410360i$ to 10^{-7} – 10^{-8} , confirming `NIntegrate` independently.
- **CUBA Vegas** (adaptive importance-sampling MC): an unrelated Monte Carlo that converges on the heavy-tailed small-coefficient integrand where plain uniform sampling fails, agreeing within 1σ .

Suite-wide CUBA cross-check (`test_cuba.wl`).

The script `EXAMPLES/test_cuba.wl` integrates *every* convergent integral of the validation suite with CUBA Cuhre applied directly to the original integrand — Test 1 ($A = 2, 3$), Test 2 (complex exponent), Test 5 (five representative kinematic points), Test 6 (Cases A/B/C, plus a lifted-MC re-check of Case A, which equals Test 23B’s integral), Test 7 (3D and 4D), and the lifted-test integrals 23A and 23D (with their exact values 5×10^5 and 10^{-6}) — and gates the tropical sector sum (1%; 2% for 4D, 5% for the PG2 case 6C) and the tropical MC (5σ) against it. Test 6B’s plain MC is reported without a gate (heavy tail from the 10^{-4} constant term, cf. Example 20). Test 3v2 contains only negative tests (nothing to integrate).

Measured at the time of writing: **all 16 cases pass**. Sector sums agree with Cuhre to 4×10^{-6} – 4×10^{-4} (1.3×10^{-3} for the PG2 case 6C); gated MCs land at 0.05 – 1.9σ . Cuhre also pins down reference values for the extreme-coefficient cases (Test 6A = 7.850075×10^{-4} , 6B = 144.0578, 6C = 1.684581×10^{-3}) and resolves both lifted-test geometries — Test 23A’s boundary sliver (5×10^5 to 2×10^{-8} relative) and Test 23D’s 10^{-6} -wide peak (10^{-6} to 10^{-10}).

The `CC/` directory contains cross-validation of the *original* package against **FIESTA**, an independent sector decomposition code used in the physics community: a $\pi/8$ convergent integral (agreement to 6 decimal places), a divergent Gamma-function integral (pole matches exactly), and a full 5D cosmological integral (bubblepp presector 1; results match FIESTA within 2σ of the MC error, with $1/\sqrt{N}$ error scaling verified from $N = 5 \times 10^4$ to 10^7).

These results predate v2. Scripts in `CC/` that call the removed divergent/IBP functions (`IdentifyDivergences`, `ProcessDivergentSector`, `EvaluateTropicalMCIBP`, ...) are retained as historical artifacts and are *not* runnable against v2.

12 Limitations and Caveats

1. **Convergent integrals only.** v2 rejects ε -regulated specifications (`noregulator`) and divergent sectors (`divergentinput`). For pole extraction use the original `TROPICAL-MONTE-CARLO` package.

2. **Polynomial coefficients.** The default and best-conditioned case is real positive coefficients, for which $\log(P_j)$ is real-valued and the cleared $Q_k \geq c_{\text{dom}} > 0$ estimate of Section 7.4 holds exactly. *Complex* coefficients are also supported — both directly (the C++ pipeline carries all polynomial values as `std::complex<double>`) and through the lifting module, which splits a small complex coefficient into a real anchor $z_0 = |C|^{1/k}$ and an $\mathcal{O}(1)$ phase residual (Sections 5.2.2, 10). The one proviso is the branch of $\log Q_k$: results use the principal branch, which is the correct, `NIntegrate`-consistent value provided no Q_k winds around 0 on $[0, 1]^n$ (guaranteed when the dominant constant term keeps Q_k near the positive real axis, as for a small complex perturbation of a positive polynomial). Genuinely complex integrands otherwise arise only from the exponents A_i and B_j .
3. **Monte Carlo convergence and heavy tails.** Accuracy scales as $1/\sqrt{N}$. When coefficients span many orders of magnitude, some sectors become heavy-tailed: the MC estimate *and its sample error bar* can both be too small at moderate N (the rare high-magnitude region is never sampled; see Example 20). Use `CheckFlatteningMagnitude` to locate such sectors and the lifting module to mitigate them.
4. **Lifting restrictions.** Lifting requires real polynomial exponents B_j (`liftcomplex` otherwise). When the lifted polynomial has too few monomials, its Newton polytope is lower-dimensional and the automatic fan fails (`liftdegenerate`); supply an explicit "FanData". Lifted sectors with `HasConstantTerm -> False` may have infinite true variance, making sample error bars unreliable (see SUMMARY.txt §8f). `DetectExtremeCoefficients` skips symbolic (kinematic) coefficients, whose magnitude is unknown at lift time.
5. **Polymake dependency.** External tool, not bundled; required only for computing fans (`tropical_fan.wl`). With `$SkipPolymakeLoad = True` and explicit `fanData`, the evaluation pipeline runs without it.
6. **Newton polytope stability.** If kinematic parameters can cause coefficients to vanish, the Newton polytope changes and the fan must be recomputed. The code assumes fixed polytope structure across all kinematic points.
7. **C++ compiler requirements.** g++ with C++17 required. OpenMP optional but recommended. macOS clang may require fallback to single-threaded mode.
8. **Double precision.** All C++ computation uses 64-bit doubles. Higher precision would require additional libraries.
9. **Memory scaling.** Results scale linearly with the number of kinematic points. Very large scans ($\gg 10,000$ points) may require attention to memory.

13 Mathematical Properties and Why It Works

Why tropical decomposition works.

As $\mathbf{x} \rightarrow 0$ in a direction $\mathbf{w}$, the polynomial $P(\mathbf{x})$ is dominated by the monomial whose exponent vector minimizes the dot product with $\mathbf{w}$. The tropical fan exactly partitions $\mathbb{R}^n$ into regions where each monomial dominates. Within each sector, the dominant monomial is factored out, leaving a polynomial $Q(\mathbf{y})$ bounded away from zero.

Why flattening is important.

Without flattening, the integrand contains factors $y_i^{a_i-1}$ which can be highly peaked near $y_i = 0$ when a_i is small. Uniform MC sampling wastes most samples away from the peak, leading to enormous variance. Flattening absorbs the y^{a-1} singularity into the integration measure, converting the integrand to $\mathcal{O}(1)$ and dramatically reducing MC variance.

Why log-exp evaluation is stable.

For $y \in (0, 1]$, $\log(y)$ varies smoothly in $(-\infty, 0]$. The exponential $\exp(\alpha \log y) = y^\alpha$ handles fractional and complex α without branch cuts or sign ambiguities. As $y \rightarrow 0^+$, the function decays or grows depending on $\operatorname{Re}(\alpha)$, but always smoothly.

Why tropical factoring is exact.

The identity $P(\mathbf{y}) = \mathbf{y}^d \cdot Q(\mathbf{y})$ is algebraic, not a numerical approximation. Every monomial in Q has non-negative exponents by construction. The constant term in Q ensures $Q \geq \text{const} > 0$ on $[0, 1]^n$. No numerical error is introduced.

Why lifting is exact.

Replacing $C \mathbf{x}^\alpha \rightarrow z^k \mathbf{x}^\alpha$ and inserting $\int dz \delta(z - z_0)$ with $z_0 = |C|^{1/k}$ changes nothing: evaluating the lifted polynomial at $z = z_0$ reproduces the original polynomial identically (the round-trip is checked symbolically by `LiftCoefficients`; residual coefficients $c_i = C_i/z_0^{k_i}$ preserve exactness when several monomials are lifted against one anchor). The delta constraint is then resolved *analytically* per sector — the pivot substitution is a monomial change of variables, not an approximation — so the lifted sector sum equals the original integral exactly. Sectors whose delta root lies outside the unit cube genuinely contribute zero (`EmptyDomain`). What lifting changes is only the *geometry* of the decomposition: the extreme coefficient becomes part of the Newton polytope of the lifted polynomial, so the fan adapts to it and the per-sector integrands are closer to $\mathcal{O}(1)$, reducing MC variance.